



UNIVERSITÉ ABDELMALEK ESSAÂDI

Faculté des Sciences et Techniques — Tanger

DÉPARTEMENT D'INFORMATIQUE

Big Data & Data Engineering

Projet : Analyse des avis des clients en temps
réel sur des produits Amazon

Auteurs :

ZARQI EZZOUBAIR

EL GHARIB MAHMOUD

EL ATTAR MOHAMED

BELKASMI ABDERRAHMANE

Encadrant :

Pr. Yasyn EL YUSUFI

Module : Big Data & Data Engineering

Filière : Master — Intelligence Artificielle & Data Science

Année : 2025 – 2026

Table des matières

1	Introduction Générale	5
1.1	Contexte et Motivation	5
1.2	Problématique	5
1.3	Objectifs du Projet	6
1.4	Données Utilisées	6
1.5	Architecture Générale	7
2	Exploration des Données	8
2.1	Introduction	8
2.2	Description du Jeu de Données	8
2.2.1	Structure Générale	8
2.2.2	Variable Cible (Label)	9
2.3	Qualité des Données	9
2.3.1	Valeurs Manquantes	9
2.3.2	Doublons	9
2.3.3	Incohérences de Helpfulness	9
2.4	Distribution des Scores et des Sentiments	9
2.4.1	Distribution des Notes (1–5 Étoiles)	10
2.4.2	Distribution des Classes de Sentiment	11
2.5	Analyse Temporelle	11
2.5.1	Évolution du Nombre d’Avis par Année	12
2.5.2	Sentiments par Année	12
2.5.3	Score Moyen par Année	13
2.6	Analyse du Contenu Textuel	13
2.6.1	Longueur des Textes	13
2.6.2	Mots les Plus Fréquents par Sentiment	15
2.7	Analyse de l’Utilité des Avis (Helpfulness)	15
2.8	Analyse des Produits	16
2.8.1	Produits les Plus Évalués	16

2.8.2	Scoring du Produit B001E4KFG0	17
2.9	Analyse des Utilisateurs	17
2.10	Synthèse des Observations	18
3	Prétraitement des Données	19
3.1	Introduction	19
3.2	Chargement et Nettoyage des Données Brutes	19
3.3	Création de la Variable Cible	20
3.4	Partitionnement Stratifié	20
3.5	Pipeline de Prétraitement NLP	20
3.5.1	Tokenisation (étape 1)	21
3.5.2	Mots vides à la carte (étape 2)	21
3.5.3	Lemmatisation et verb-first (étape 3)	21
3.5.4	Bigrammes et combinaison (étapes 4-5)	21
3.5.5	HashingTF et IDF (étapes 6-7)	22
3.6	Gestion du Déséquilibre de Classes	22
3.7	Sauvegarde des Artefacts	22
3.8	Synthèse	23
4	Expérimentation des Modèles	24
4.1	Vue d'ensemble	24
4.2	Stratégie Expérimentale	24
4.3	Classifieurs Évalués	24
4.4	Données et Features	25
4.5	Résultats sur le Jeu de Validation	25
4.5.1	Analyse par Modèle	25
4.6	Critère de Sélection du Modèle	26
4.7	Analyse Visuelle	26
4.8	Principaux Résultats	26
5	Entraînement du Modèle Final	29
5.1	Vue d'ensemble	29
5.2	Préparation des Données	29
5.3	Poids de Classe Amplifiés	30
5.4	Pipeline NLP	30
5.5	Modèle de Référence	30
5.6	Réglage des Hyperparamètres	30
5.6.1	Stratégie de Recherche	30
5.6.2	Résultats	31
5.7	Évaluation du Modèle Optimisé	31

5.8	Calibration des Seuils de Décision	32
5.9	Artefacts Sauvegardés	32
5.10	Du Notebook au Job Spark de Production	32
5.11	Évaluation Finale sur le Jeu de Test	34
5.11.1	Résultats Globaux	34
5.11.2	Performances par Classe	34
5.11.3	Matrice de Confusion	35
5.11.4	Traçabilité	35
6	Architecture et Environnement Technique	37
6.1	Vue d'ensemble	37
6.2	Conteneurisation avec Docker	38
6.2.1	Principe de déploiement	38
6.2.2	Gestion des permissions	38
6.2.3	Architecture interne de Spark	38
6.3	Services Déployés	39
6.4	Apache Spark	40
6.4.1	Image personnalisée	40
6.4.2	Cluster Standalone	41
6.4.3	Jobs Spark	41
6.5	Apache Kafka et ZooKeeper	41
6.6	MongoDB	41
6.7	Apache Airflow	41
6.7.1	Image personnalisée	42
6.7.2	Configuration	42
6.8	Flux de Données Global	42
6.9	Récapitulatif des Technologies	42
7	Architecture d'Implémentation et Orchestration	44
7.1	Vue d'ensemble	44
7.2	Stratégie de Soumission au Cluster Spark	44
7.3	Philosophie d'Implémentation et Traçabilité	44
7.4	Le Producteur Kafka : Simulation du Flux Temps Réel	45
7.5	Jobs de Préparation et d'Apprentissage (Batch)	45
7.5.1	Job de Prétraitement	45
7.5.2	Job d'Entraînement	46
7.5.3	Job d'Évaluation	46
7.6	Job d'Inférence en Flux Continu (Streaming)	46
7.7	Orchestration des Pipelines avec Apache Airflow	47
7.7.1	Pipeline d'Entraînement End-to-End	47

7.7.2	Agrégation Nocturne pour le Tableau de Bord	47
7.7.3	Surveillance du Modèle et Détection de Dérive	47
8	Intégration Système et Déploiement Local	49
8.1	Vue d'ensemble	49
8.2	Architecture Conteneurisée et Réseau	49
8.3	Couche d'Exposition des Données (API)	49
8.4	Validation Fonctionnelle	50
9	Résultats et Validation du Système	51
9.1	Infrastructure Conteneurisée	51
9.2	Orchestration Airflow	51
9.3	Suivi Spark	52
9.4	Persistance MongoDB	52
9.5	Tableau de Bord	53
10	Conclusion	59
10.1	Bilan du Projet	59
10.2	Limites Identifiées	60
10.3	Perspectives d'Amélioration	60

Introduction Générale

1.1 Contexte et Motivation

Dans un monde de plus en plus connecté, les plateformes de commerce électronique génèrent chaque jour des millions d’avis clients qui constituent une source d’information précieuse pour les entreprises. Amazon, leader mondial du e-commerce, héberge à lui seul des centaines de milliers de commentaires sur ses produits alimentaires, reflétant les expériences et les opinions d’une clientèle diverse et internationale.

L’analyse de ces avis présente cependant des défis majeurs : leur volume considérable, leur vélocité de génération et la variété de leur contenu rendent toute approche manuelle impossible. C’est dans ce contexte que s’inscrit le présent projet, qui vise à concevoir et déployer une architecture de traitement de données en temps réel capable d’analyser automatiquement le sentiment exprimé dans les avis clients.

1.2 Problématique

Face à un flux continu de commentaires textuels, comment est-il possible d’extraire automatiquement et en temps réel la polarité émotionnelle — positive, négative ou neutre — de chaque avis ? Cette question soulève plusieurs sous-problèmes techniques :

- **Le traitement du langage naturel** : les textes bruts nécessitent une chaîne de pré-traitement rigoureuse (tokenisation, lemmatisation, vectorisation TF-IDF) avant de pouvoir être soumis à un modèle d’apprentissage automatique.
- **Le déséquilibre des classes** : la distribution des avis est fortement asymétrique, avec une majorité d’avis positifs, ce qui constitue un obstacle majeur à la performance des modèles de classification.
- **Le traitement en temps réel** : l’analyse doit s’effectuer sur un flux continu de

données avec une latence minimale, ce qui exige une architecture distribuée et résiliente.

- **La visualisation et l'archivage** : les résultats de prédiction doivent être persistés, agrégés et présentés à travers un tableau de bord interactif permettant une analyse historique.

1.3 Objectifs du Projet

Ce projet a pour objectif principal la conception d'un système complet d'analyse de sentiment en temps réel sur les avis clients Amazon. Plus précisément, les objectifs sont les suivants :

1. Construire une pipeline de traitement de données en temps réel reposant sur les technologies **Apache Kafka** et **Apache Spark Streaming**.
2. Développer et évaluer plusieurs modèles de classification de sentiment (*Logistic Regression, Naive Bayes, Random Forest*) sur le jeu de données Amazon Fine Food Reviews, et sélectionner le modèle le plus performant.
3. Déployer le modèle retenu au sein de la pipeline de streaming pour effectuer des prédictions en continu sur les avis entrants.
4. Archiver les résultats dans une base de données **MongoDB** et les exposer via un tableau de bord *off-line* développé avec **FastAPI** et **Chart.js**.
5. Conteneuriser l'ensemble de l'architecture avec **Docker** afin de garantir la reproductibilité et la portabilité du système.

1.4 Données Utilisées

Le jeu de données utilisé dans ce projet est le *Amazon Fine Food Reviews* disponible sur la plateforme Kaggle¹. Il contient environ **568 454 avis** collectés sur une période de plus de dix ans, incluant l'identifiant du produit, le profil de l'utilisateur, la note attribuée (de 1 à 5 étoiles) et le contenu textuel de l'avis.

La variable cible est définie à partir de la note selon la règle suivante :

- **Négatif** : note < 3
- **Neutre** : note = 3
- **Positif** : note > 3

1. <https://www.kaggle.com/snap/amazon-fine-food-reviews>

1.5 Architecture Générale

L'architecture du système s'articule autour de cinq composants principaux, comme illustré dans la figure 6.1 :

1. **Producteur Kafka** : simule le flux temps réel en publiant les avis du jeu de test dans le topic `reviews.raw`.
2. **Cluster Kafka** : assure la collecte, la distribution et la persistance du flux de messages à travers ses brokers et ses partitions.
3. **Spark Streaming** : consomme le flux Kafka, applique la chaîne de prétraitement NLP et réalise les prédictions de sentiment à l'aide du modèle de régression logistique entraîné.
4. **MongoDB** : archive les résultats de prédiction pour une consultation ultérieure et alimente le tableau de bord.
5. **Tableau de bord FastAPI** : présente les résultats sous forme de visualisations interactives, notamment l'évolution temporelle des prédictions et la répartition des sentiments par produit.

Exploration des Données

2.1 Introduction

Le jeu de données *Amazon Fine Food Reviews* [?] rassemble 568 454 avis de produits alimentaires publiés entre 1999 et 2012. Cette exploration vise à cartographier la structure, la qualité et les distributions des variables avant le prétraitement et la modélisation.

2.2 Description du Jeu de Données

2.2.1 Structure Générale

Le fichier `Reviews.csv` contient **568 454 lignes** et **10 colonnes** pour une empreinte mémoire d'environ 467,4 Mo. Le tableau 2.1 détaille chaque colonne.

Colonne	Type	Description
Id	int64	Identifiant unique de l'avis
ProductId	object	Identifiant unique du produit
UserId	object	Identifiant unique de l'utilisateur
ProfileName	object	Nom (pseudo) de l'utilisateur
HelpfulnessNumerator	int64	Nombre de votes n utile z reçus
HelpfulnessDenominator	int64	Nombre total de votes reçus
Score	int64	Note de 1 à 5 étoiles
Time	int64	Horodatage Unix
Summary	object	Résumé court de l'avis
Text	object	Contenu complet de l'avis

TABLE 2.1. Description des colonnes du jeu de données *Amazon Fine Food Reviews*

2.2.2 Variable Cible (Label)

La cible de sentiment est construite à partir de Score :

- › **Négatif (0)** : Score < 3
- › **Neutre (1)** : Score = 3
- › **Positif (2)** : Score > 3

2.3 Qualité des Données

2.3.1 Valeurs Manquantes

Seuls ProfileName (26 valeurs) et Summary (27 valeurs) présentent des manques, tous négligeables (moins de 0,01 %). Ces lignes seront simplement supprimées lors du prétraitement.

2.3.2 Doublons

Aucune ligne n'est exactement dupliquée, mais **174 875 textes** (30,76 %) apparaissent plusieurs fois. Ce taux élevé de doublons dans le champ Text nécessitera un filtrage pour éviter de biaiser l'entraînement des modèles.

2.3.3 Incohérences de Helpfulness

Deux lignes seulement violent l'inégalité $\text{HelpfulnessNumerator} \leq \text{HelpfulnessDenominator}$; elles seront écartées lors du nettoyage.

2.4 Distribution des Scores et des Sentiments

2.4.1 Distribution des Notes (1–5 Étoiles)

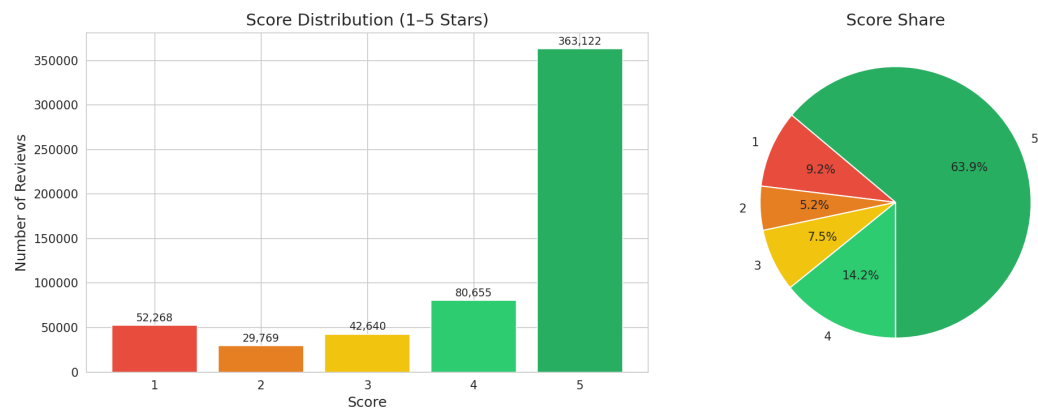


FIGURE 2.1. Distribution des notes de 1 à 5 étoiles et proportion de chaque note.

La figure 2.1 montre une asymétrie massive en faveur des notes élevées. La note 5 étoiles concentre 63,9 % des avis (363 122 entrées), suivie de la note 4 étoiles (14,2 %). Les notes négatives (1 et 2 étoiles) ne totalisent que 14,4 % (tableau 2.2). Le score moyen s'établit à 4,18 étoiles.

Score	Nombre d'avis	Proportion
1	52 268	9,2%
2	29 769	5,2%
3	42 640	7,5%
4	80 655	14,2%
5	363 122	63,9%
Total	568 454	100%

TABLE 2.2. Répartition des notes dans le jeu de données

2.4.2 Distribution des Classes de Sentiment

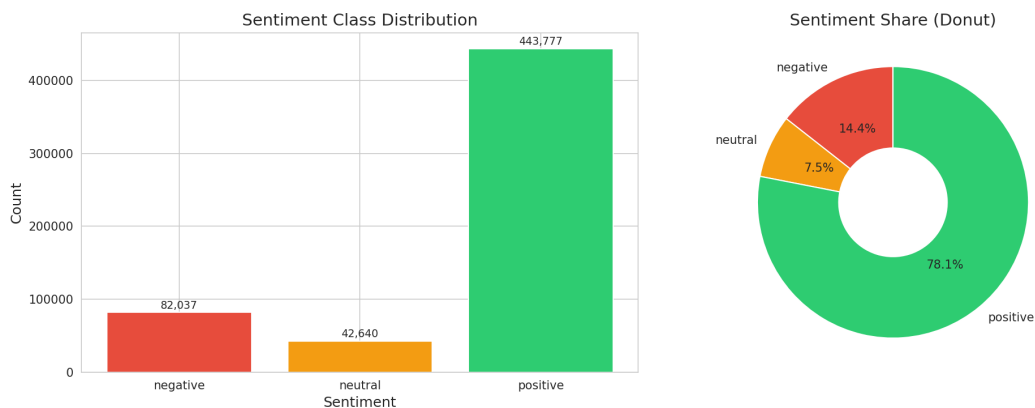


FIGURE 2.2. Distribution des classes de sentiment (négatif, neutre, positif).

La figure 2.2 et le tableau 2.3 confirment un **déséquilibre de classes** marqué : 78,1 % d'avis positifs, contre 14,4 % de négatifs et seulement 7,5 % de neutres. Ce déséquilibre devra être compensé par une pondération adaptée lors de l'entraînement.

Sentiment	Nombre d'avis	Proportion
Négatif	82 037	14,4%
Neutre	42 640	7,5%
Positif	443 777	78,1%
Total	568 454	100%

TABLE 2.3. Répartition des classes de sentiment

2.5 Analyse Temporelle

2.5.1 Évolution du Nombre d'Avis par Année

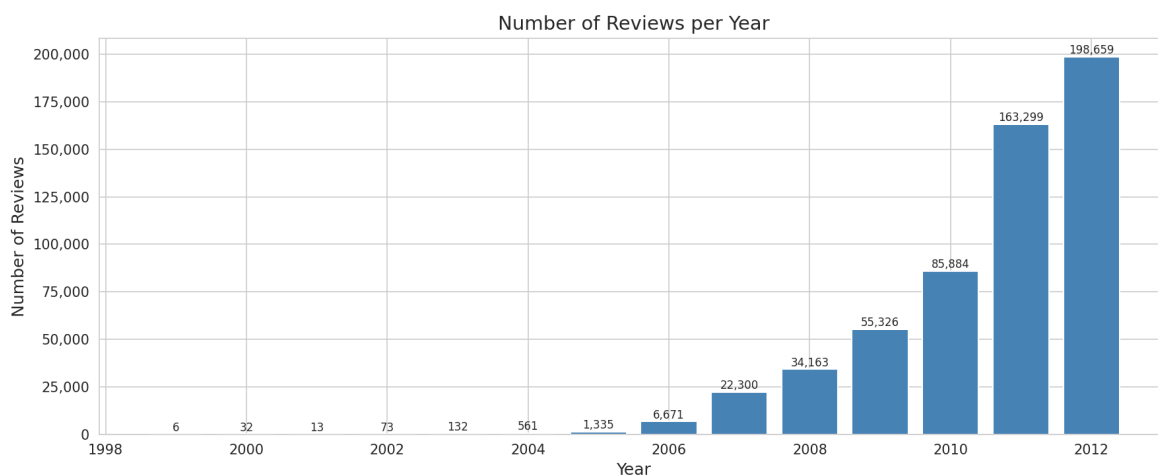


FIGURE 2.3. Nombre d'avis par année (1999–2012).

La figure 2.3 montre une croissance exponentielle du volume d'avis, passant de quelques centaines avant 2004 à près de 200 000 avis en 2012. L'année 2012 est la plus représentée avec **198 659 avis**.

2.5.2 Sentiments par Année

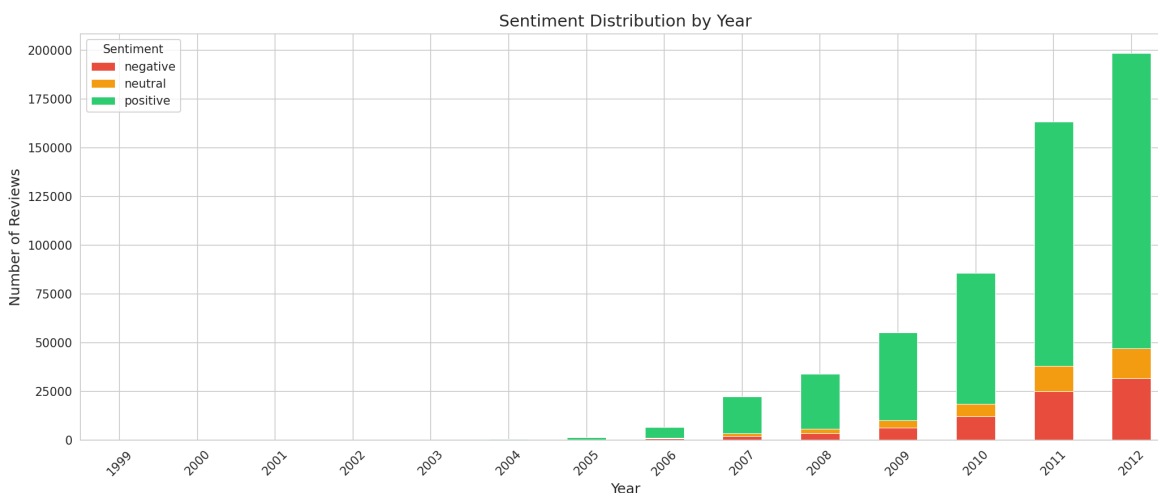


FIGURE 2.4. Distribution des sentiments par année (avis empilés).

La figure 2.4 montre que la proportion d'avis positifs reste dominante sur toute la période. La part des avis négatifs augmente légèrement à partir de 2010, peut-être le reflet d'une communauté plus critique.

2.5.3 Score Moyen par Année

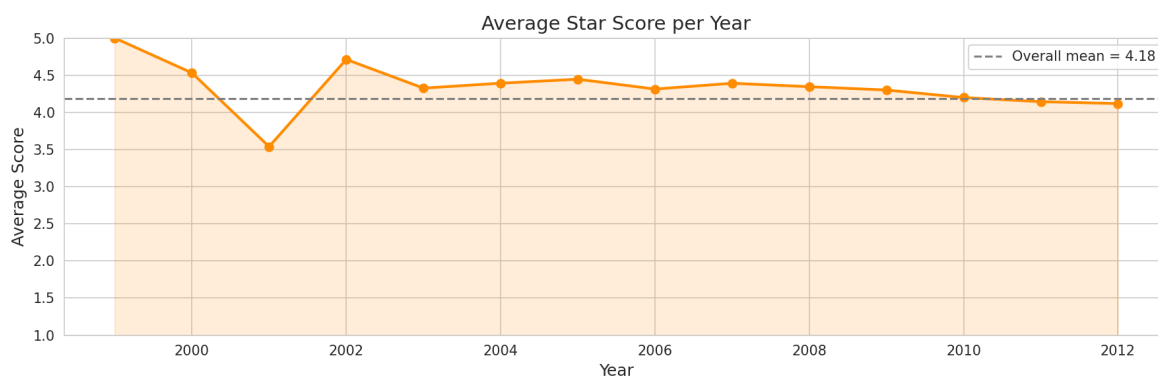


FIGURE 2.5. Score moyen par année avec la moyenne globale (4,18 étoiles).

Le score moyen annuel oscille entre 4,10 et 4,50 étoiles, avec une moyenne globale de **4,18 étoiles** (figure 2.5). Les fluctuations plus fortes avant 2003 s'expliquent par le faible nombre d'avis.

2.6 Analyse du Contenu Textuel

2.6.1 Longueur des Textes

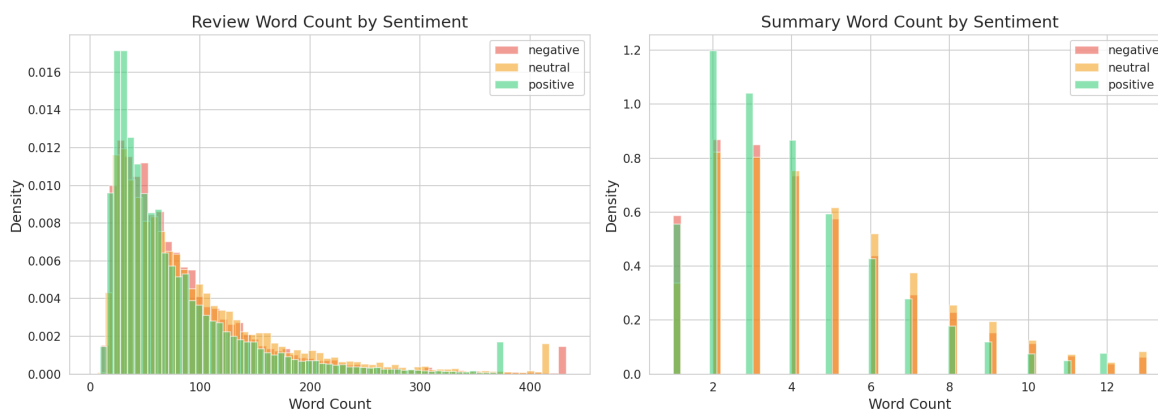


FIGURE 2.6. Distribution du nombre de mots dans les textes et les résumés par sentiment.

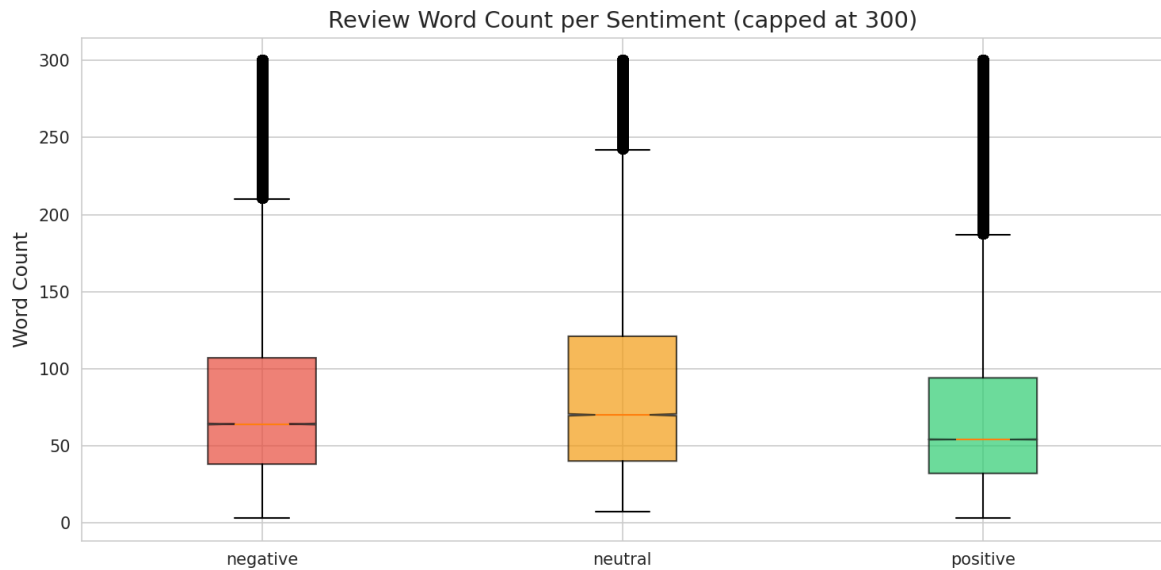


FIGURE 2.7. Boîtes à moustaches du nombre de mots par sentiment (plafonné à 300).

Les textes comptent en moyenne 80 mots, avec une médiane à 56 mots et une forte asymétrie à droite (maximum 3 432 mots). Les résumés sont beaucoup plus courts (2 à 6 mots en moyenne). La figure 2.6 et le tableau 2.4 montrent que les avis négatifs sont légèrement plus longs que les positifs (médiane 65 contre 53 mots), suggérant que les utilisateurs mécontents s’expriment davantage.

Statistique	Valeur
Observations	568 427
Moyenne	80,3 mots
Écart-type	79,5 mots
Minimum	3 mots
Q1	33 mots
Médiane	56 mots
Q3	98 mots
Maximum	3 432 mots

TABLE 2.4. Statistiques descriptives de la longueur des textes (en mots)

2.6.2 Mots les Plus Fréquents par Sentiment

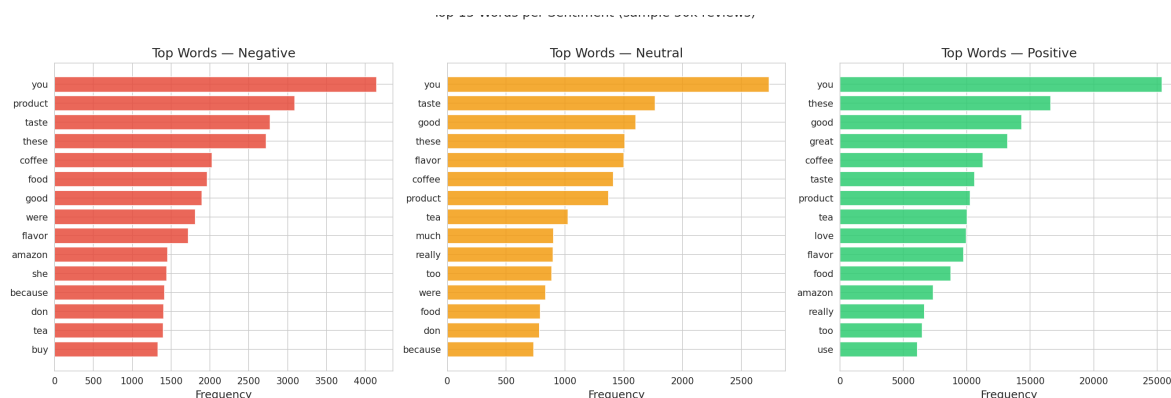


FIGURE 2.8. Top 15 des mots les plus fréquents par classe de sentiment (échantillon de 50 000 avis).

La figure 2.8 révèle des champs lexicaux distincts. Dans les avis négatifs, *product*, *taste*, *buy* et *amazon* reviennent souvent, évoquant des plaintes sur la qualité ou la livraison. Les avis positifs sont dominés par *great*, *good*, *love* et *delicious*. La classe neutre emprunte un vocabulaire plus mitigé.

2.7 Analyse de l'Utilité des Avis (Helpfulness)

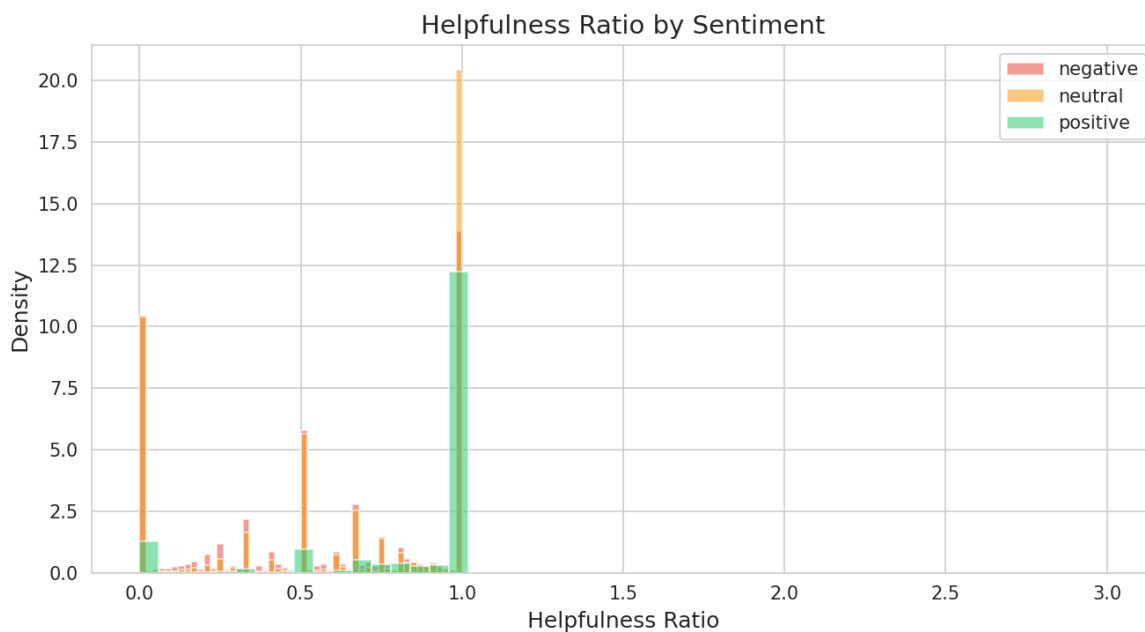


FIGURE 2.9. Distribution du ratio d'utilité (*helpfulness ratio*) par sentiment.

Le ratio $r = \frac{\text{HelpfulnessNumerator}}{\text{HelpfulnessDenominator}}$ montre deux modes à $r = 0$ (inutile) et $r = 1$ (totalement utile). Les avis positifs affichent un pic plus marqué à $r = 1$, ce qui indique qu'ils sont perçus comme plus utiles par la communauté (figure 2.9).

2.8 Analyse des Produits

2.8.1 Produits les Plus Évalués

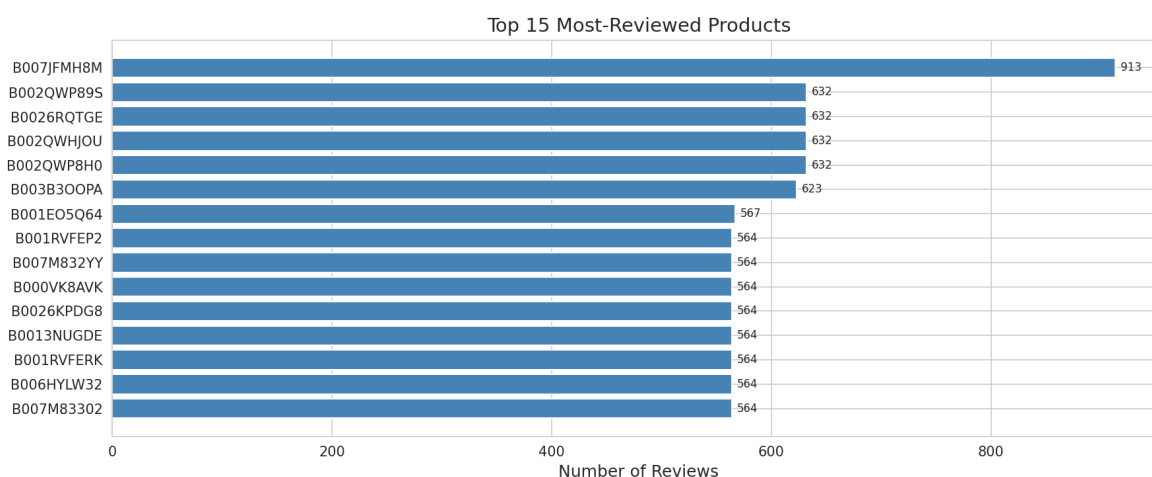


FIGURE 2.10. Top 15 des produits les plus évalués.

Le produit **B007JFMH8M** domine avec 913 avis. Plusieurs autres produits (B002QWP89S, B0026RQTGE, B002QWHJOU, B002QWP8H0) en comptent chacun 632. Au total, le jeu contient **74 258 produits uniques**.

2.8.2 Scoring du Produit B001E4KFG0

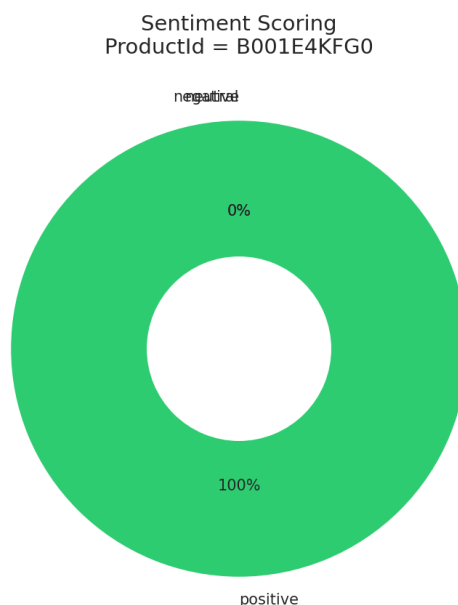


FIGURE 2.11. Répartition des sentiments pour le produit ProductId = B001E4KFG0.

Le produit **B001E4KFG0** (*Good Quality Dog Food*) reçoit exclusivement des avis positifs (figure 2.11).

2.9 Analyse des Utilisateurs

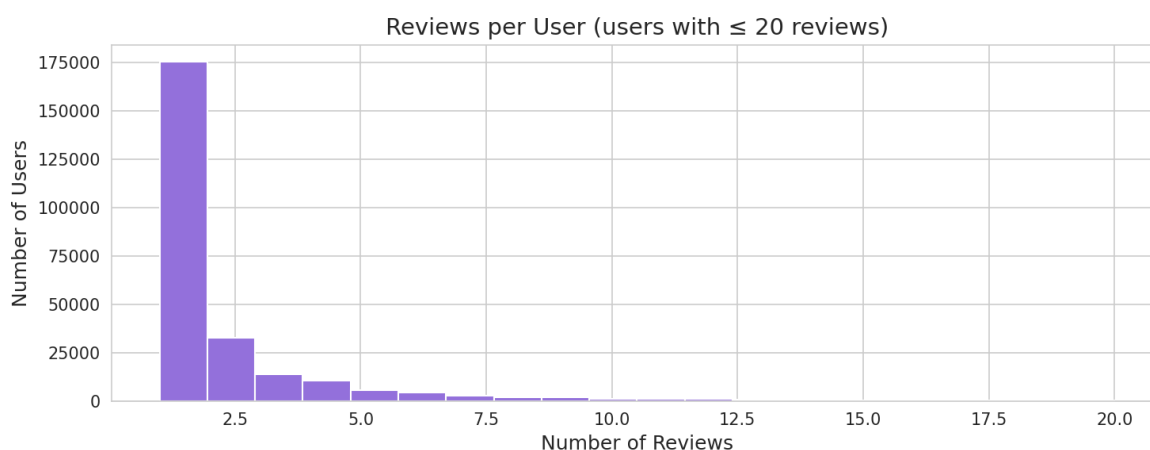


FIGURE 2.12. Distribution du nombre d'avis par utilisateur (utilisateurs avec ≤ 20 avis).

La distribution du nombre d'avis par utilisateur suit une loi de puissance : la grande majorité (**plus de 175 000 utilisateurs**) ne publie qu'un seul avis, tandis qu'un

tout petit nombre en produit une grande quantité. L'utilisateur le plus prolifique a déposé 448 avis (identifiant A30XHLG6DIBRW8). Au total, le jeu compte **256 059 utilisateurs uniques** (figure 2.12).

2.10 Synthèse des Observations

Le tableau 2.5 résume les points clés qui guideront les étapes suivantes (suppression des doublons, pondération des classes, vectorisation avec prise en compte de la négation).

Observation	Détail
Taille du jeu de données	568 454 avis, 10 colonnes
Déséquilibre des classes	Positif 78,1%, Neutre 7,5%, Négatif 14,4%
Ratio positif/neutre	$\approx 10 : 1$ nécessite une pondération ou un ré-échantillonnage
Période temporelle	10+ ans (1999–2012), croissance exponentielle
Valeurs manquantes	Très faibles (ProfileName et Summary uniquement)
Textes dupliqués	30,76% de doublons dans le champ Text
Longueur des textes	Grande variance ; les avis négatifs tendent à être plus longs
Mots clés par sentiment	Négatifs : <i>bad, waste, taste</i> Positifs : <i>great, love, best</i>
Score moyen	4,18 étoiles (stable dans le temps)

TABLE 2.5. Synthèse des observations de l'analyse exploratoire

Prétraitement des Données

3.1 Introduction

Le pipeline de prétraitement transforme les données brutes *Amazon Fine Food Reviews* en vecteurs exploitables par les classifieurs. Construit avec PySpark, il respecte une règle stricte de non-fuite : le preprocesseur est ajusté uniquement sur l'entraînement, puis appliqué tel quel à la validation et au test.

Le pipeline génère les partitions brutes (80/10/10) et leurs versions vectorisées : `data/train`, `data/val`, `data/test`, ainsi que `data/trainfeat`, `data/valfeat`, `data/testfeat`.

3.2 Chargement et Nettoyage des Données Brutes

Le fichier `reviews.csv` est chargé avec inférence de schéma (568 454 lignes). Quatre nettoyages sont appliqués dans la foulée :

1. Suppression des lignes sans texte (Text nul ou vide).
2. Déduplication des textes identiques pour éviter la fuite entre partitions.
3. Filtrage des lignes où `HelpfulnessNumerator` > `HelpfulnessDenominator`.
4. Remplacement des résumés (Summary) absents par une chaîne vide.

Ces opérations éliminent 180 153 lignes, ramenant le jeu à **388 301** lignes (tableau 3.1).

État	Nombre de lignes
Données brutes	568 454
Après nettoyage	388 301
Lignes supprimées	180 153

TABLE 3.1. Effet du nettoyage sur la taille du jeu de données

3.3 Création de la Variable Cible

La cible est dérivée du Score : négatif (Score < 3), neutre (Score = 3), positif (Score > 3). La distribution après nettoyage est donnée dans le tableau 3.2 (78,3% de positifs, 14,2% de négatifs, 7,5% de neutres).

Sentiment	Label	Nombre d'avis	Proportion
Négatif	0	55 222	14,2%
Neutre	1	29 127	7,5%
Positif	2	303 952	78,3%
Total	—	388 301	100%

TABLE 3.2. Distribution des classes après nettoyage

3.4 Partitionnement Stratifié

Le jeu est découpé en trois parts (80/10/10) par stratification sur la cible, garantissant que chaque classe conserve la même proportion dans chaque partie (figure 3.1). Les effectifs obtenus sont listés dans le tableau 3.3.

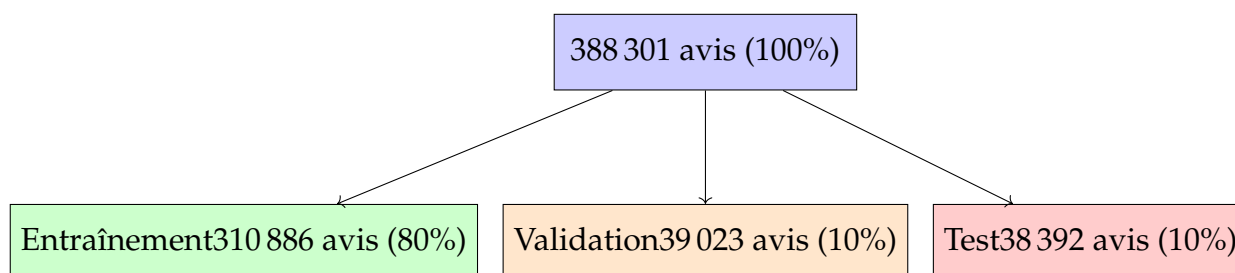


FIGURE 3.1. Partitionnement stratifié 80/10/10

Sentiment	Train	Validation	Test
Positif	243 060	30 661	30 231
Négatif	44 350	5 509	5 363
Neutre	23 476	2 853	2 798
Total	310 886	39 023	38 392

TABLE 3.3. Distribution des classes dans chaque partition

3.5 Pipeline de Prétraitement NLP

Le cur du prétraitement est un pipeline PySpark à sept étapes résumé dans le tableau 3.4.

Étape	Composant	Entrée	Sortie
1	RegexTokenizer	Text	words
2	StopWordsRemover	words	filtered
3	LemmatizerTransformer	filtered	lemmatized
4	NGram ($n = 2$)	lemmatized	bigrams
5	UnigramBigramCombiner	lemmatized + bigrams	tokens
6	HashingTF	tokens	rawfeatures
7	IDF	rawfeatures	features

TABLE 3.4. Étapes du pipeline NLP

3.5.1 Tokenisation (étape 1)

Un `RegexTokenizer` avec le motif `\w+` extrait les séquences alphanumériques et écarte la ponctuation, en ignorant les tokens de moins de 2 caractères.

3.5.2 Mots vides à la carte (étape 2)

La liste de stop words par défaut de PySpark contient 181 mots, dont des négations essentielles pour le sentiment. Une liste personnalisée de 178 mots préserve *not*, *no*, *nor*, *never*, *neither*, *none*, *nobody*, *nowhere*, *hardly*, *barely*, *scarcely* (tableau 3.5). Ainsi, un segment comme *í not good ž* n'est pas réduit à *í good ž*.

Liste	Nombre de mots
Liste par défaut (PySpark)	181
Liste personnalisée	178
Négations préservées	11

TABLE 3.5. Comparaison des listes de mots vides

3.5.3 Lemmatisation í verb-first ž (étape 3)

Un transformateur sur mesure enrobe le `WordNetLemmatizer` de NLTK. La lemmatisation tente d'abord la forme verbale ($pos='v'$) puis la nominale ($pos='n'$) : *loved* $\rightarrow$ *love*, *batteries* $\rightarrow$ *battery*. Si les deux échouent, le token est mis en minuscule. Cette approche aide à regrouper les variantes morphologiques d'un même sentiment.

3.5.4 Bigrammes et combinaison (étapes 4-5)

Les bigrammes (`NGram`, $n = 2$) capturent des expressions locales comme *not_good* ou *highly_recommend*. Un `UnigramBigramCombiner` fusionne unigrammes et bigrammes en une seule séquence, offrant au modèle à la fois le lexique et le contexte.

3.5.5 HashingTF et IDF (étapes 6-7)

La projection dans un espace de hachage de **100 000 dimensions** (HashingTF) réduit les collisions par rapport à un espace plus petit, tout en restant économe en mémoire. Le IDF atténue ensuite les termes trop fréquents et ignore les tokens apparaissant dans moins de 3 documents (`minDocFreq=3`).

3.6 Gestion du Déséquilibre de Classes

Des poids de classe sont calculés à partir de l'inverse des fréquences relatives dans l'entraînement (tableau 3.6). Ils seront passés à la colonne `classWeight` des modèles pour pénaliser davantage les erreurs sur les classes minoritaires. La classe neutre reçoit ainsi le poids le plus élevé (4,41), tandis que la positive est atténuée (0,43).

Classe	Label	Effectif (Train)	Poids
Négatif	0	44 350	2.3366
Neutre	1	23 476	4.4142
Positif	2	243 060	0.4264

TABLE 3.6. Poids de classe pour la gestion du déséquilibre

3.7 Sauvegarde des Artefacts

Seuls les étages **stateful** du pipeline (HashingTF et IDF) sont sauvegardés en tant que modèles, car ce sont eux qui gardent une mémoire des données d'entraînement. Les partitions brutes et vectorisées sont également conservées pour réutilisation immédiate (tableau 3.7). Les transformateurs sans paramètres (tokeniseur, lemmatiseur, etc.) seront reconstruits à l'identique dans les jobs ultérieurs.

Artefact	Chemin
Partition d'entraînement brute	data/train
Partition de validation brute	data/val
Partition de test brute	data/test
Partition d'entraînement vectorisée	data/trainfeat
Partition de validation vectorisée	data/valfeat
Partition de test vectorisée	data/testfeat
Modèle HashingTF (fitted)	data/models/hashingtff
Modèle IDF (fitted)	data/models/idf

TABLE 3.7. Artefacts produits par le pipeline de prétraitement

3.8 Synthèse

Le pipeline garantit l'absence de fuite, préserve la négation, combine unigrammes et bigrammes sur un large espace de hachage (100 k dimensions) et pondère les classes pour compenser le déséquilibre. Les partitions vectorisées sont maintenant prêtes pour l'entraînement et la comparaison des modèles de classification.

Expérimentation des Modèles

4.1 Vue d'ensemble

Ce chapitre compare quatre classifieurs sur les données prétraitées *Amazon Fine Food Reviews* afin d'identifier le meilleur candidat pour un réglage ultérieur. Tous les modèles sont entraînés avec PySpark MLlib sur la partition d'entraînement (`trainfeat`) et évalués sur la partition de validation (`valfeat`, 39 023 exemples). Le jeu de test n'a jamais été consulté lors de cette phase.

4.2 Stratégie Expérimentale

Les partitions vectorisées sont chargées depuis Parquet, la colonne `classWeight` est rattachée, puis quatre classifieurs sont entraînés avec leurs hyperparamètres par défaut (en incluant les poids de classe). Chaque modèle est évalué sur la validation via le F1 macro global, la précision, et les F1 par classe. Les matrices de confusion et les profils de performance guident la sélection du meilleur modèle pour le réglage des hyperparamètres.

4.3 Classifieurs Évalués

- **Régression Logistique** : modèle linéaire multinomial avec régularisation ℓ_2 (`regParam = 0,01`, 100 itérations).
- **SVM Linéaire (One-vs-Rest)** : classifieur linéaire rapide, adapté aux données creuses de grande dimension; les poids de classe sont appliqués via le wrapper `OneVsRest`.
- **Naive Bayes (Multinomial)** : classifieur probabiliste très rapide, mais reposant sur l'hypothèse d'indépendance conditionnelle des features – hypothèse forte pour des vecteurs TF-IDF corrélés. L'implémentation PySpark ne propose pas de `weightCol`.

- **Forêt Aléatoire** : méthode d'ensemble non linéaire incluse à titre de référence (`numTrees = 10`, `maxDepth = 5`). Son comportement dégénéré sur features TF-IDF la disqualifie (Section 4.5).

4.4 Données et Features

L'entraînement compte 310 886 avis sous forme de vecteurs TF-IDF creux (0 = négatif, 1 = neutre, 2 = positif). Les poids de classe inverses (Tableau 4.1) compensent partiellement le fort déséquilibre, notamment la rareté de la classe neutre.

TABLE 4.1. Distribution des classes et poids dans le jeu d'entraînement

Label	Sentiment	Effectif	Poids
0	Négatif	44 350	2,34
1	Neutre	23 476	4,41
2	Positif	243 060	0,43

4.5 Résultats sur le Jeu de Validation

Le tableau 4.2 rassemble les performances des quatre modèles. Les modèles linéaires dominent largement.

TABLE 4.2. Performances des modèles sur le jeu de validation

Modèle	F1	Préc.	F1 _{neg}	F1 _{neu}	F1 _{pos}	Train (s)
SVM Linéaire (OvR)	0,8065	0,8014	0,5896	0,2444	0,8977	78,8
Régression Logistique	0,8038	0,7940	0,5916	0,2558	0,8929	51,7
Naïve Bayes	0,8002	0,7788	0,6043	0,3004	0,8819	2,9
Forêt Aléatoire	0,6914	0,7857	0,0000	0,0000	0,8800	270,9

4.5.1 Analyse par Modèle

La **Forêt Aléatoire** prédit quasi systématiquement la classe majoritaire (positif), ce qui donne un F1 nul sur les classes négative et neutre. Les vecteurs TF-IDF creux et de haute dimension sont inadaptés au sous-échantillonnage aléatoire des features de l'algorithme.

Naïve Bayes obtient le meilleur F1 neutre (0,3004) et un entraînement éclair (2,9 s), mais son hypothèse d'indépendance conditionnelle est violée dans du texte naturel TF-IDF où les termes corrélerent. Son score compétitif ne peut donc pas être interprété de manière fiable.

Le **SVM Linéaire** décroche le meilleur F1 global (0,8065) et la meilleure précision, avec un excellent F1 positif (0,8977). Cependant, il accuse le plus mauvais score sur la classe neutre (0,2444), le maillon faible de tous les classifieurs.

La **Régression Logistique** suit de très près le SVM en F1 global (0,8038) tout en offrant un F1 neutre un peu supérieur (0,2558) et un temps d'entraînement raisonnable (51,7 s). En tant que modèle discriminatif, elle ne souffre pas des limitations de Naive Bayes.

4.6 Critère de Sélection du Modèle

Pour équilibrer performance globale et traitement de la classe minoritaire, nous utilisons un score combiné pondéré :

$$\text{Score} = 0,70 \times F1_{\text{macro}} + 0,30 \times F1_{\text{neutre}} \quad (4.1)$$

TABLE 4.3. Classement final selon le score combiné (Équation 4.1)

Modèle	F1 _{macro}	F1 _{neutre}	Score Combiné
Naive Bayes	0,8002	0,3004	0,6503
Régression Logistique	0,8038	0,2558	0,6394
SVM Linéaire (OvR)	0,8065	0,2444	0,6379
Forêt Aléatoire	0,6914	0,0000	0,4840

Même si Naive Bayes affiche le meilleur score combiné, son hypothèse d'indépendance est incompatible avec les dépendances inhérentes aux vecteurs TF-IDF. **La Régression Logistique** est donc retenue comme meilleur modèle valide pour le réglage des hyperparamètres : elle est robuste aux features corrélées, offre un bon compromis entre F1 global (0,8038), F1 neutre (0,2558) et coût d'entraînement (51,7 s).

4.7 Analyse Visuelle

4.8 Principaux Résultats

La classe neutre reste le point faible de tous les classifieurs, confirmant la nécessité des poids de classe et d'un réglage fin. Les modèles linéaires surclassent nettement les arbres sur cette représentation TF-IDF creuse. Naive Bayes est écarté malgré un bon F1 neutre, car son hypothèse d'indépendance est structurellement inadaptée. La Régression Logistique, sélectionnée pour le réglage, offre un équilibre solide entre performance globale, récupération neutre et temps d'entraînement.

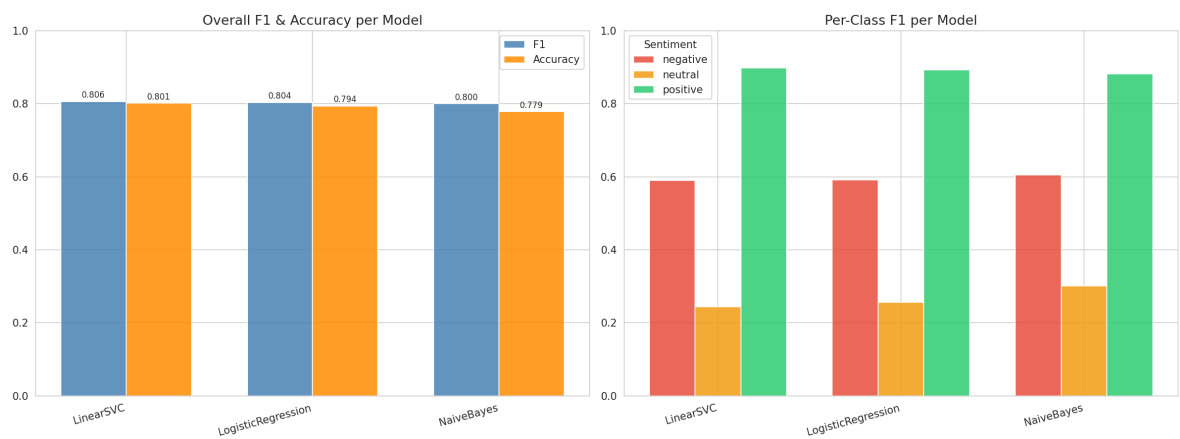


FIGURE 4.1. F1 global et précision par modèle (gauche); scores F1 par classe pour les sentiments négatif, neutre et positif (droite).

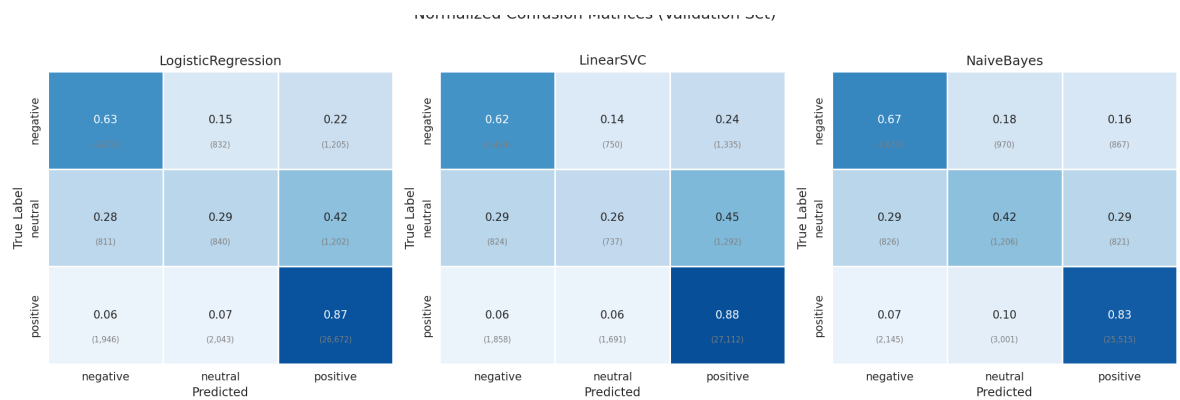


FIGURE 4.2. Matrices de confusion des quatre modèles évalués sur le jeu de validation. La Forêt Aléatoire se concentre entièrement sur la classe positive, tandis que la Régression Logistique offre le meilleur équilibre global.

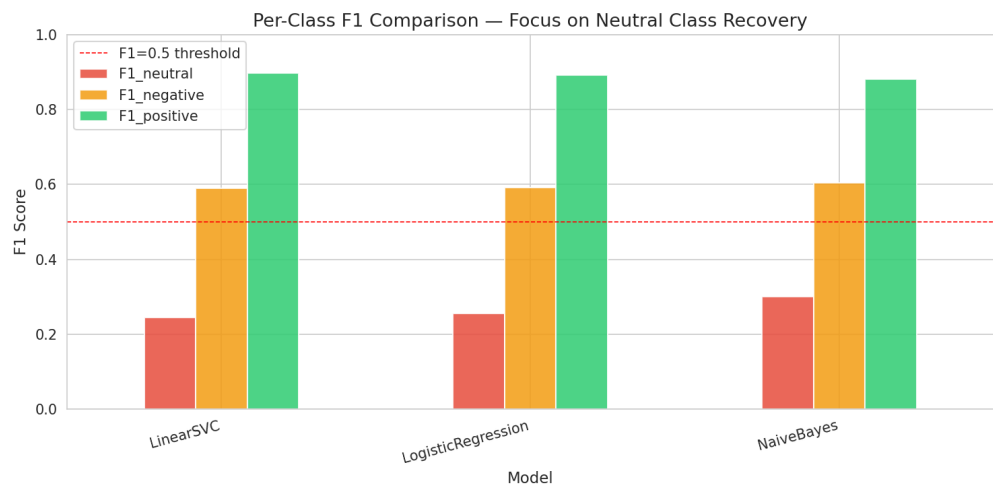


FIGURE 4.3. Comparaison du F1 par classe, avec focus sur la récupération de la classe neutre. La ligne rouge pointillée marque le seuil $F1 = 0.5$. Tous les modèles restent en dessous de ce seuil sur la classe neutre, soulignant la difficulté de la tâche.

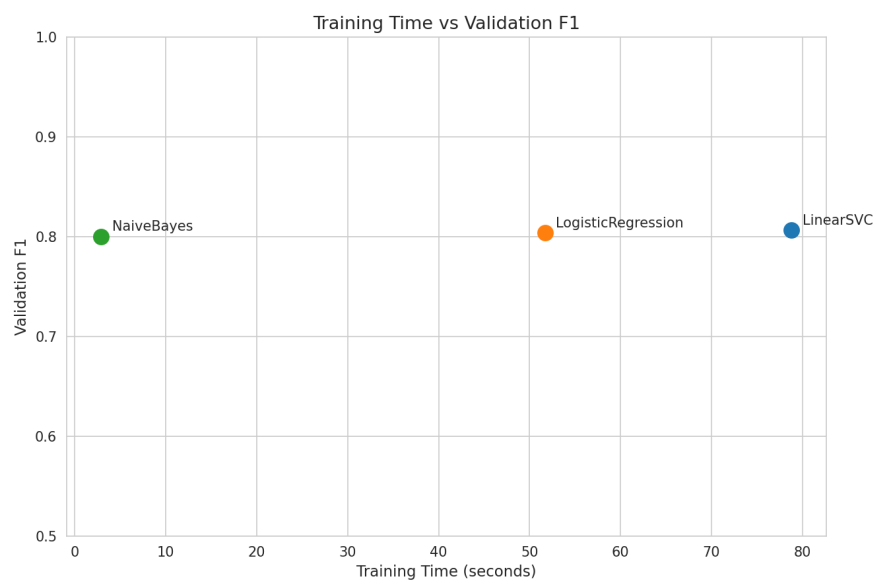


FIGURE 4.4. Compromis temps d'entraînement / F1 de validation. La Régression Logistique offre un bon équilibre entre performance et coût computationnel. La Forêt Aléatoire (à droite) est à la fois la plus lente et la moins précise.

Entraînement du Modèle Final

5.1 Vue d'ensemble

Le modèle final, une **Régression Logistique Multinomiale**, reprend le meilleur classifieur du chapitre précédent et l'enrichit de quatre améliorations majeures : des poids de classe amplifiés, une fusion des résumés (*Summary*) avec le corps du texte, un espace de features étendu à 100 000 dimensions (bigrammes inclus) et un réglage fin par `TrainValidationSplit`. Le jeu de validation est strictement réservé au réglage, le test n'étant jamais consulté.

TABLE 5.1. Améliorations par rapport à l'expérimentation initiale

Aspect	Expérimentation	Ce notebook
Poids de classe	Fréquence inverse standard	Amplifiés (neutre= 8,0, négatif= 3,0)
Features	<i>Text</i> uniquement	<i>Summary</i> $\times 2$ + <i>Text</i>
numFeatures	10 000	100 000
N-grammes	Unigrammes	Unigrammes + bigrammes
Réglage	Aucun	<code>TrainValidationSplit</code> (12 combinaisons)
Seuil de décision	$\arg \max(\text{prob})$	Seuils calibrés par classe

5.2 Préparation des Données

Les partitions CSV d'entraînement (310 886 avis) et de validation (39 023) sont chargées. On concatène deux fois le *Summary* avec le *Text* (`Summary || Summary || Text`) afin de surpondérer le résumé, court mais très informatif, dans le calcul TF-IDF.

5.3 Poids de Classe Amplifiés

Pour forcer l'attention sur les classes minoritaires, les poids inverses de fréquence sont remplacés par des valeurs manuelles alignées sur le critère de sélection ($0,70 \times F1_{\text{macro}} + 0,30 \times F1_{\text{neutre}}$).

TABLE 5.2. Poids de classe amplifiés

Classe	Label	Poids
Négatif	0	3,0
Neutre	1	8,0
Positif	2	0,5

Le poids de 8,0 sur la classe neutre pénalise lourdement ses erreurs, tandis que le poids 0,5 pour la classe positive limite sa domination naturelle.

5.4 Pipeline NLP

Le pipeline textuel, appliqué uniquement sur l'entraînement, enchaîne : tokenisation par expression régulière, suppression des mots vides en préservant les négations (not, no, never), lemmatisation *nlp* verb-first *z*, extraction de bigrammes, fusion unigrammes/bigrammes, hachage TF sur 100 000 dimensions et pondération IDF avec *minDocFreq*=3. L'ajustement prend environ 50,8 s.

5.5 Modèle de Référence

Avant tout réglage, une première régression (régularisation ℓ_2 pure, *regParam*=0,01, *maxIter*=100) est entraînée pour mesurer l'apport des nouvelles features.

TABLE 5.3. Performances du modèle de référence (validation)

Modèle	F1	Préc.	F1 _{neg}	F1 _{neu}	F1 _{pos}
Baseline	0,8374	0,8281	0,6684	0,3248	0,9154

Un gain de +0,0336 par rapport au chapitre précédent confirme l'efficacité de la fusion Summary et des bigrammes. L'entraînement dure 47,5 s.

5.6 Réglage des Hyperparamètres

5.6.1 Stratégie de Recherche

On utilise *TrainValidationSplit* (80/20 sur *trainfeat*) plutôt qu'une validation croisée complète pour réduire le temps de calcul. La grille couvre 12 combinaisons de

regParam, elasticNetParam et maxIter (Tableau 5.4). La métrique est le F1 pondéré global. La recherche dure 929,3 s.

TABLE 5.4. Grille d'hyperparamètres

Paramètre	Valeurs	Justification
regParam	0,0001 ; 0,001 ; 0,01	Moins de régularisation pour les minoritaires
elasticNetParam	0,0 (ℓ_2) ; 0,2 (ElasticNet)	Parcimonie ElasticNet
maxIter	100 ; 200	Convergence sur la classe neutre

5.6.2 Résultats

Le meilleur score (**0,8482**) est obtenu avec regParam = 0,001, elasticNetParam = 0,2 et maxIter = 100. ElasticNet apporte une légère parcimonie qui filtre mieux les features TF-IDF non informatives.

TABLE 5.5. Scores F1 des 12 combinaisons

#	regParam	elasticNet	maxIter	F1
01	0,0001	0,0	100	0,8302
02	0,0001	0,0	200	0,8286
03	0,0001	0,2	100	0,8426
04	0,0001	0,2	200	0,8426
05	0,001	0,0	100	0,8317
06	0,001	0,0	200	0,8317
07	0,001	0,2	100	0,8482
08	0,001	0,2	200	0,8481
09	0,01	0,0	100	0,8374
10	0,01	0,0	200	0,8374
11	0,01	0,2	100	0,8163
12	0,01	0,2	200	0,8163

5.7 Évaluation du Modèle Optimisé

Sur le jeu de validation complet (39 023 exemples, seuil par défaut), le modèle atteint F1= 0,8473 et précision= 0,8349, avec un gain de +0,0990 sur le F1 neutre par rapport au benchmark.

TABLE 5.6. Performances optimisées (validation)

Modèle	F1	Préc.	F1 _{neg}	F1 _{neu}	F1 _{pos}
LR optimisée	0,8473	0,8349	0,6996	0,3548	0,9197

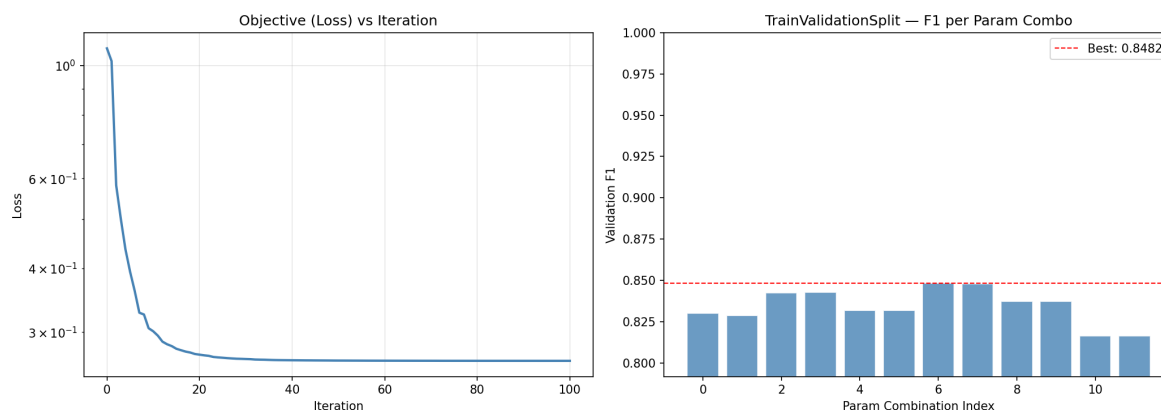


FIGURE 5.1. Gauche : convergence de la perte avant l'itération 60. Droite : F1 de validation interne des 12 combinaisons (meilleur en rouge).

5.8 Calibration des Seuils de Décision

Pour favoriser le rappel de la classe neutre, on cherche les seuils (τ_{neg} , τ_{neu} , τ_{pos}) qui maximisent le score combiné $0,70 \times F1_{\text{macro}} + 0,30 \times F1_{\text{neutre}}$. Les valeurs optimales sont $\tau_{\text{neg}} = 0,35$, $\tau_{\text{neu}} = 0,22$, $\tau_{\text{pos}} = 0,40$ (score 0,6892). La Table 5.7 montre un léger recul du F1 global compensé par un meilleur équilibre précision/rappel sur le neutre.

TABLE 5.7. Impact de la calibration (validation)

Configuration	F1	Préc.	F1 _{neg}	F1 _{neu}	F1 _{pos}
Sans calibration	0,8473	0,8349	0,6996	0,3548	0,9197
Avec calibration	0,8358	0,8152	0,6684	0,3248	0,9154

5.9 Artefacts Sauvegardés

Seuls les modèles `LogisticRegressionModel`, `HashingTFModel` et `IDFModel` sont persistés sur disque, accompagnés du fichier JSON de seuils calibrés ($\text{neg}=0,35$, $\text{neu}=0,22$, $\text{pos}=0,40$). Le pipeline de transformation textuel, sans état, est reconstruit à la volée dans le job de streaming.

5.10 Du Notebook au Job Spark de Production

Les notebooks sont parfaits pour l'exploration, mais fragiles en production. La logique d'entraînement a donc été extraite dans un script autonome (`train_lr_job.py`) exécuté via `spark-submit`. Ce job déclare explicitement la `SparkSession`, configure manuellement les partitions de shuffle et applique les hyperparamètres optimaux trouvés précédemment.

Listing 5.1. Extrait de train_lr_job.py

```

1 from pyspark.ml.classification import LogisticRegression
2 from pyspark.sql import SparkSession
3
4 spark = (SparkSession.builder
5         .appName("AmazonReviews-BestLR-Training")
6         .config("spark.sql.shuffle.partitions", "8")
7         .getOrCreate())
8 spark.sparkContext.setLogLevel("WARN")
9
10 train_feat = spark.read.parquet(DATA_DIR + "/train_feat")
11 lr = LogisticRegression(
12     featuresCol="features", labelCol="label",
13     weightCol="classWeight", family="multinomial",
14     regParam=0.001, elasticNetParam=0.2, maxIter=100)
15 lr_model = lr.fit(train_feat)

```

```

26/05/09 18:06:47 INFO ResourceUtils: =====
26/05/09 18:06:47 INFO SparkContext: Submitted application: AmazonReviews-BestLR-Training
26/05/09 18:06:47 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(memory -> name: memory, amount: 4096, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ),
  task resources: Map(cpus -> name: cpus, amount: 1.0)
26/05/09 18:06:47 INFO ResourceProfile: Limiting resource is cpu
26/05/09 18:06:47 INFO ResourceProfileManager: Added ResourceProfile id: 0
26/05/09 18:06:47 INFO SecurityManager: Changing view acls to: spark
26/05/09 18:06:47 INFO SecurityManager: Changing modify acls to: spark
26/05/09 18:06:47 INFO SecurityManager: Changing view acls groups to:
26/05/09 18:06:47 INFO SecurityManager: Changing modify acls groups to:
26/05/09 18:06:47 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: spark; groups with view permissions: EMPTY; users with modify permissions: spark; groups with modify permissions: EMPTY
26/05/09 18:06:48 INFO Utils: Successfully started service 'sparkDriver' on port 46545.
26/05/09 18:06:48 INFO SparkEnv: Registering MapOutputTracker
26/05/09 18:06:48 INFO SparkEnv: Registering BlockManagerMaster
26/05/09 18:06:48 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopologyMapper for getting topology information
26/05/09 18:06:48 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
26/05/09 18:06:48 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/05/09 18:06:48 INFO DiskBlockManager: Created local directory at /tmp/blockmgr-98b1a204-e942-4269-8581-697899a60900
26/05/09 18:06:48 INFO MemoryStore: MemoryStore started with capacity 2.2 GiB
26/05/09 18:06:48 INFO SparkEnv: Registering OutputCommitCoordinator
26/05/09 18:06:48 INFO JettyUtils: Start Jetty 0.0.0.0:4040 for SparkUI
26/05/09 18:06:48 INFO Utils: Successfully started service 'SparkUI' on port 4040.
26/05/09 18:06:48 INFO StandaloneAppClient$ClientEndpoint: Connecting to master spark://spark-master:7077...
26/05/09 18:06:48 INFO TransportClientFactory: Successfully created connection to spark-master/172.24.0.3:7077 after 20 ms (0 ms spent in bootstraps)
26/05/09 18:06:48 INFO StandaloneSchedulerBackend: Connected to Spark cluster with app ID app-20260509180648-0000
26/05/09 18:06:48 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 35663.
26/05/09 18:06:48 INFO NettyBlockTransferService: Server created on e7e21925cb5e:35663
26/05/09 18:06:48 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
26/05/09 18:06:48 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, e7e21925cb5e, 35663, None)
26/05/09 18:06:48 INFO BlockManagerMasterEndpoint: Registering block manager e7e21925cb5e:35663 with 2.2 GiB RAM, BlockManagerId(driver, e7e21925cb5e, 35663, None)
26/05/09 18:06:48 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, e7e21925cb5e, 35663, None)
26/05/09 18:06:48 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, e7e21925cb5e, 35663, None)
26/05/09 18:06:48 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20260509180648-0000/0 on worker-20260509175649-172.24.0.6:33695 (172.24.0.6:33695) with 8 core(s)
26/05/09 18:06:48 INFO StandaloneSchedulerBackend: Granted executor ID app-20260509180648-0000/0 on hostPort 172.24.0.6:33695 with 8 core(s), 4.0 GiB RAM
26/05/09 18:06:48 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20260509180648-0000/0 is now RUNNING
26/05/09 18:06:48 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.0
✓ Spark 3.5.1 ready.
Loading train features...
Train features: 454,754 rows
Training Logistic Regression.
26/05/09 18:07:06 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
✓ Training done in 226.9s
26/05/09 18:10:43 WARN TaskSetManager: Stage 363 contains a task of very large size (2399 KIB). The maximum recommended task size is 1000 KIB.
✓ LR model saved - /opt/spark/work-dir/data/output/models/best_lr
✓ Training metadata saved - /opt/spark/work-dir/data/output/models/training_meta.json
✓ SparkSession stopped. Training job complete.

```

FIGURE 5.2. Logs du job d'entraînement Spark

La reproductibilité est assurée par un fichier de métadonnées (training_meta.json) enregistrant l'horodatage, la version et les hyperparamètres figés.

Listing 5.2. Génération des métadonnées

```

1 import json
2 from datetime import datetime, timezone
3
4 meta = {

```

```

5     "trained_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT
      %H:%M:%SZ"),
6     "model_name": "LogisticRegression",
7     "model_version": "v1.0.0",
8     "hyperparameters": {
9         "regParam": lr_model.getRegParam(),
10        "elasticNetParam": lr_model.getElasticNetParam(),
11        "maxIter": lr_model.getMaxIter(),
12        "numFeatures": 100_000
13    }
14 }
15 with open(META_PATH, "w") as f: json.dump(meta, f, indent=2)

```

5.11 Évaluation Finale sur le Jeu de Test

Le job `evaluate_lr_job.py` applique le modèle sauvegardé au jeu de test (56 847 lignes, jamais consultées auparavant). Les métriques sont calculées en mode distribué par `MulticlassClassificationEvaluator`.

5.11.1 Résultats Globaux

Avec une accuracy de 0,8910 et un F1 pondéré de 0,8939, le modèle dépasse nettement les scores de validation (Tableau 5.8). L'écart s'explique par la taille plus importante du test et par la capacité de généralisation apportée par l'enrichissement des features et les poids de classe amplifiés.

TABLE 5.8. Performances globales sur le jeu de test (56 847 lignes)

Métrique	Score
Accuracy	0,8910
F1 Pondéré	0,8939
Précision Pondérée	0,8974
Rappel Pondéré	0,8910

5.11.2 Performances par Classe

Le Tableau 5.9 montre une très bonne tenue sur le négatif (F1 = 0,7860) et une progression remarquable sur le neutre (F1 = 0,5545), fruit des poids amplifiés et de la préservation des négations.

TABLE 5.9. Métriques par classe (test)

Classe	Précision	Rappel	F1
Négatif	0,7720	0,8006	0,7860
Neutre	0,5158	0,5994	0,5545
Positif	0,9578	0,9362	0,9469

5.11.3 Matrice de Confusion

La confusion principale se situe entre les classes neutre et positive (1 570 faux positifs, 979 faux neutres, Tableau 5.10). C'est ce que la calibration des seuils vise à atténuer en production.

TABLE 5.10. Matrice de confusion (test, lignes = vrai, colonnes = prédit)

	Prédit Négatif	Prédit Neutre	Prédit Positif
Vrai Négatif	6 695	822	846
Vrai Neutre	724	2 548	979
Vrai Positif	1 253	1 570	41 410

```

26/05/09 18:11:04 INFO MemoryStore: MemoryStore started with capacity 1048.0 MiB
26/05/09 18:11:04 INFO SparkEnv: Registering OutputCommitCoordinator
26/05/09 18:11:04 INFO JettyUtils: Start Jetty 9.4.0.v20140404 for SparkUI
26/05/09 18:11:04 INFO Utils: Successfully started service 'SparkUI' on port 4040.
26/05/09 18:11:04 INFO StandaloneClientEndpoint: Connecting to master spark://spark-master:7077...
26/05/09 18:11:04 INFO TransportClientFactory: Successfully created connection to spark-master/172.24.0.3:7077 after 19 ms (0 ms spent in bootstraps)
26/05/09 18:11:04 INFO StandaloneSchedulerBackend: Connected to Spark cluster with app ID app-20260509181104-0000
26/05/09 18:11:04 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 36129.
26/05/09 18:11:04 INFO NettyBlockTransferService: Server created on 95cb3c7d11f136129
26/05/09 18:11:04 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
26/05/09 18:11:04 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, 95cb3c7d11f1, 36129, None)
26/05/09 18:11:04 INFO BlockManagerMasterEndpoint: Registering block manager 95cb3c7d11f136129 with 1048.0 MiB RAM, BlockManagerId(driver, 95cb3c7d11f1, 36129, None)
26/05/09 18:11:04 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, 95cb3c7d11f1, 36129, None)
26/05/09 18:11:04 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, 95cb3c7d11f1, 36129, None)
26/05/09 18:11:04 INFO StandaloneClientEndpoint: Executor added: app-20260509181104-0000/0 on worker-20260509175649-172.24.0.6:33695 (172.24.0.6:33695) with 8 core(s)
26/05/09 18:11:04 INFO StandaloneSchedulerBackend: Granted executor ID app-20260509181104-0000/0 on hostPort 172.24.0.6:33695 with 8 core(s), 2.0 GiB RAM
26/05/09 18:11:05 INFO StandaloneClientEndpoint: Executor updated: app-20260509181104-0000/0 is now RUNNING
26/05/09 18:11:05 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.0
✓ Spark 3.5.1 ready.
Loading test features from /opt/spark/work-dir/data/output/test_feat...
Test rows: 56,847
Loading model from /opt/spark/work-dir/data/output/models/best_lr...
Generating predictions on test set...
Calculating metrics...
26/05/09 18:11:10 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:20 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:21 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:22 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:22 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:22 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
Weighted F1 : 0.8935
Accuracy : 0.8985
26/05/09 18:11:23 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:23 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:24 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
negative P:0.7701 R:0.8008 F1:0.7852
26/05/09 18:11:24 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:24 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
neutral P:0.5161 R:0.6043 F1:0.5587
26/05/09 18:11:25 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:25 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
26/05/09 18:11:26 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
positive P:0.9579 R:0.9358 F1:0.9463
Calculating confusion matrix...
26/05/09 18:11:26 WARN DAGScheduler: Broadcasting large task binary with size 2.4 MiB
✓ model_insights.json written ~ /opt/spark/work-dir/data/output/model_insights.json
✓ SparkSession stopped. Evaluation job complete.

```

FIGURE 5.3. Logs du job d'évaluation

5.11.4 Traçabilité

Le job écrit un fichier `model_insights.json` consolidant les métriques, la matrice de confusion et les références aux métadonnées d'entraînement. Ce rapport automatique valide la promotion du modèle en production.

Listing 5.3. Extrait de `model_insights.json`


```
1 {  
2   "evaluated_at": "2026-05-10T15:10:10Z",  
3   "model_name": "LogisticRegression",  
4   "trained_at": "2026-05-10T15:09:01Z",  
5   "metrics": {  
6     "accuracy": 0.891,  
7     "f1_weighted": 0.8939,  
8     "per_class": { ... }  
9   },  
10  "confusion_matrix": { ... }  
11 }
```

Architecture et Environnement Technique

6.1 Vue d'ensemble

Le projet s'exécute dans un environnement entièrement conteneurisé avec Docker et Docker Compose, garantissant reproductibilité, isolation et portabilité entre les machines de développement. Les services – calcul distribué, messagerie, orchestration et stockage – communiquent via un réseau Docker dédié (`amazon_reviews_network`). La figure 6.1 illustre les flux principaux.

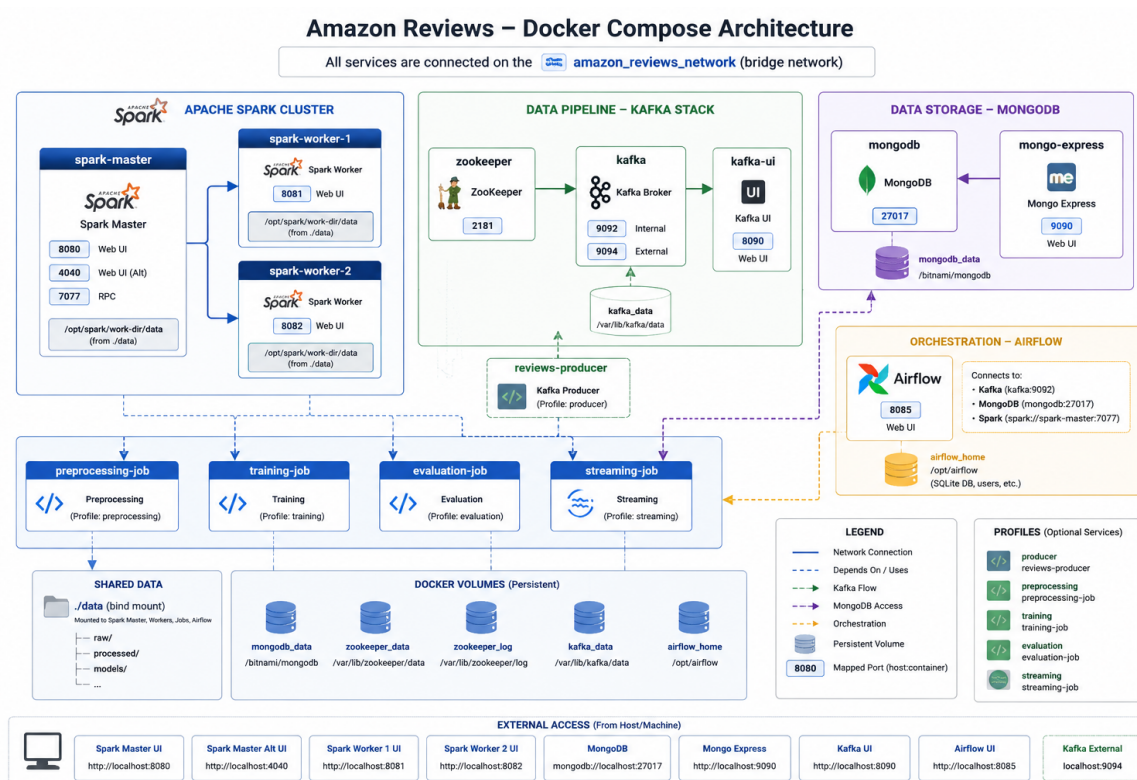


FIGURE 6.1. Vue d'ensemble de l'architecture conteneurisée.

6.2 Conteneurisation avec Docker

6.2.1 Principe de déploiement

Tous les services sont définis dans un fichier `docker-compose.yaml` unique. Les secrets et paramètres (identifiants MongoDB, configuration Kafka) sont regroupés dans un fichier `.env` partagé via une ancre YAML. Le réseau bridge permet la résolution de noms entre conteneurs sans ouvrir de ports superflus sur l'hôte.

6.2.2 Gestion des permissions

Les conteneurs Spark (UID 1000) et Airflow (UID 50000) requièrent des droits spécifiques sur les volumes partagés. Un simple `chown -R 1000:0 ./data` et un réglage des ACL sur les logs Airflow suffisent à éviter les erreurs d'accès.

6.2.3 Architecture interne de Spark

Spark adopte une organisation maître-esclave : un **driver** pilote l'exécution en découpant le travail en tâches qu'il distribue à des **executors** hébergés sur les workers. Le **Cluster Manager** (allocation des ressources) est le mode standalone de Spark, incarné par le conteneur `spark-master` (figure 6.2).

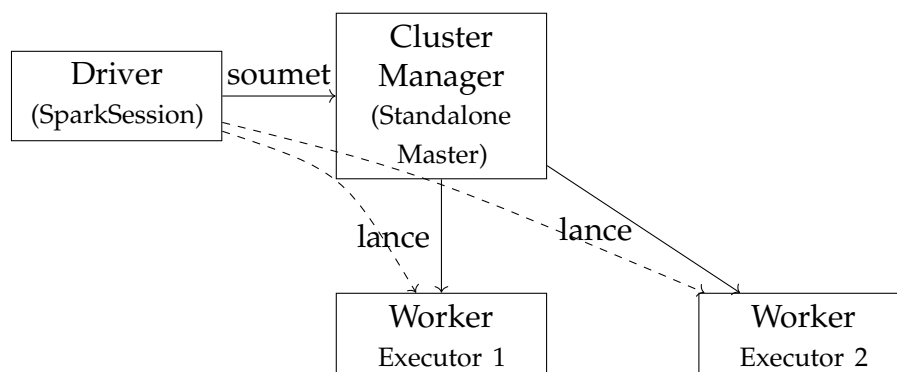


FIGURE 6.2. Architecture de Spark en cluster standalone.

Le driver (dans un conteneur de job client-mode ou dans le conteneur de streaming) soumet l'application au cluster manager, qui lance les executors. Chaque worker (`spark-worker-1`, `spark-worker-2`) héberge un executor. Pour le streaming, le flux Kafka est découpé en micro-lots traités comme des DataFrames batch. La tolérance aux pannes repose sur la journalisation des offsets Kafka et la reproductibilité des transformations.

6.3 Services Déployés

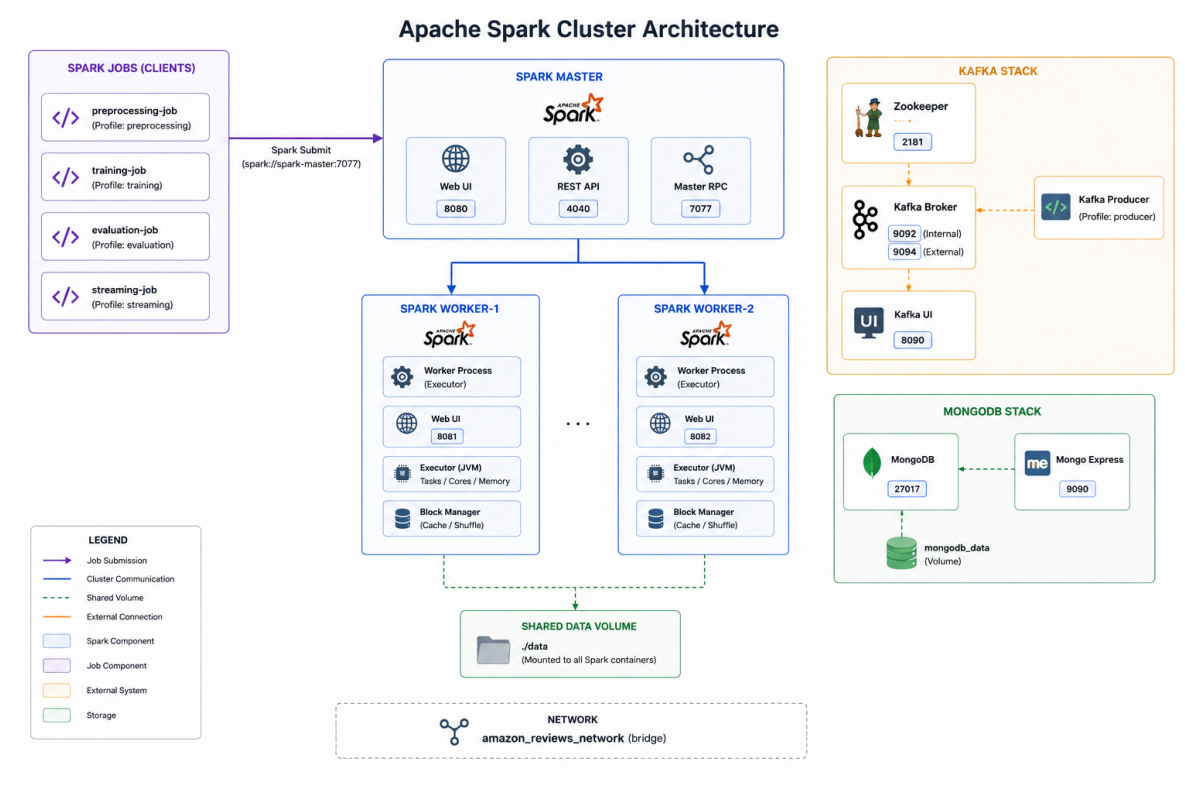


FIGURE 6.3. Schéma global de l'architecture Spark déployée.

Le tableau 6.1 dresse l'inventaire complet des conteneurs.

TABLE 6.1. Inventaire des services Docker Compose

Conteneur	Image / Build	Port(s)	Rôle
spark-master	Dockerfile.spark	8080, 7077	Maître du cluster Spark
spark-worker-1	Dockerfile.spark	8081	Worker Spark (n°1)
spark-worker-2	Dockerfile.spark	8082	Worker Spark (n°2)
mongodb	bitnami/mongodb	27017	Base NoSQL (résultats)
mongo-express	mongo-express	9090	Interface d'administration MongoDB
zookeeper	confluentinc/cp-zookeeper:7.6.0	2181	Coordinateur Kafka
kafka	confluentinc/cp-kafka:7.6.0	9092, 9094	Broker de messagerie
kafka-ui	provectuslabs/kafka-ui	8090	Supervision Kafka (web)
kafka-producer	./kafka/Dockerfile	–	Injection des avis dans reviews.raw
preprocessing-job	./spark/preprocessing_job	–	Prétraitement batch (Spark)
streaming-job	./spark/streaming_job	–	Inférence temps réel (Spark Streaming)
training-job	./spark/training_job	–	Entraînement du modèle (Spark)
airflow	Dockerfile.airflow	8085	Orchestration des workflows

6.4 Apache Spark

6.4.1 Image personnalisée

L'image Spark part de `apache/spark:3.5.1` et y ajoute les librairies Python et Java nécessaires (Listing 6.1). Les JARs Kafka pour le streaming structuré sont téléchargés au *build*.

Listing 6.1. Extrait de `Dockerfile.spark`

```

1 FROM apache/spark:3.5.1
2 RUN pip install numpy pandas scikit-learn pymongo kafka-python
   nltk
3 RUN python3 -m nltk.downloader wordnet omw-1.4
4 # JARs pour Spark Structured Streaming
5 RUN curl -o /opt/spark/jars/spark-sql-kafka-0-10_2.12-3.5.1.jar
   ...
6     curl -o /opt/spark/jars/kafka-clients-3.4.1.jar ...

```

Les quatre JARs ajoutés sont : le connecteur Spark-Kafka, son fournisseur de to-

kens, le client Kafka bas niveau et le pool de connexions Commons Pool2.

6.4.2 Cluster Standalone

Le cluster se compose d'un maître (spark-master) et de deux workers. Les workers s'enregistrent auprès du maître via `spark://spark-master:7077`. Un volume partagé `./data` est monté sur tous les nuds, rendant les données et les modèles accessibles uniformément.

6.4.3 Jobs Spark

Deux jobs principaux sont packagés en services Docker avec profils d'activation :

- **preprocessing-job** (profil preprocessing) – nettoyage, tokenisation, lemmatisation, TF-IDF et sauvegarde des partitions Parquet.
- **streaming-job** (profil streaming) – consommation en temps réel du topic `reviews.raw`, application du pipeline NLP et du modèle de Régression Logistique, écriture dans MongoDB. Le paramètre `restart: unless-stopped` assure sa résilience.

6.5 Apache Kafka et ZooKeeper

Kafka (Confluent 7.6.0) fait office de bus de messages entre le producteur d'avis et le job de streaming, avec ZooKeeper pour la coordination. Le broker expose un listener interne (`PLAINTEXT://kafka:9092`) et un externe (`localhost:9094`). La création automatique des topics est activée pour simplifier le déploiement.

Le conteneur `kafka-producer` lit le fichier CSV de test et publie chaque avis dans le topic `reviews.raw` avec un délai de 0,2s, simulant un flux temps réel. L'IHM `kafka-ui` (port 8090) permet de vérifier les topics, les consommateurs et les offsets.

6.6 MongoDB

MongoDB (image Bitnami) stocke les prédictions du streaming. Son modèle document convient aux avis enrichis (texte libre, métadonnées, label, probabilités). Les données persistent dans un volume nommé `mongodb_data`; `mongo-express` (port 9090) offre une interface web d'inspection.

6.7 Apache Airflow

6.7.1 Image personnalisée

Airflow est basé sur `apache/airflow:2.9.3-python3.10`, complété des providers Spark et MongoDB, ainsi que des clients Kafka et Mongo (Listing 6.2).

Listing 6.2. Extrait de `Dockerfile.airflow`

```
1 FROM apache/airflow:2.9.3-python3.10
2 RUN pip install \
3     apache-airflow-providers-apache-spark==4.9.0 \
4     apache-airflow-providers-mongo==4.2.0 \
5     kafka-python==2.0.2 pymongo==4.7.0 pandas==2.2.2 python-
    dotenv==1.0.0
```

6.7.2 Configuration

Airflow est déployé en mode **standalone** (exécuteur séquentiel, base SQLite), ce qui est suffisant pour un environnement de développement mono-machine. Les DAGs sont chargés depuis le volume `./airflow/dags`, et la persistance de la base est assurée par `airflow_home`. Les variables d'environnement fournissent les connexions vers Kafka et MongoDB.

6.8 Flux de Données Global

Le chemin d'une donnée est le suivant : le producteur envoie un avis dans le topic `reviews.raw`; le job Spark Structured Streaming le consomme, le transforme (tokenisation, lemmatisation, bigrammes, TF-IDF) et applique la Régression Logistique (avec calibration des seuils); le résultat est écrit dans MongoDB. Airflow orchestre l'enchaînement des jobs par des DAGs.

6.9 Récapitulatif des Technologies

TABLE 6.2. Technologies utilisées et leur rôle

Technologie	Version	Rôle	Catégorie
Apache Spark	3.5.1	Traitement batch et streaming	Calcul distribué
PySpark MLlib	3.5.1	Entraînement et inférence ML	Machine Learning
Apache Kafka	7.6.0	Messagerie événementielle	Streaming
ZooKeeper	7.6.0	Coordination Kafka	Infrastructure
MongoDB	latest	Stockage des résultats	Base de données
Apache Airflow	2.9.3	Orchestration des workflows	Orchestration
Docker Compose	–	Conteneurisation multi-service	Déploiement
Python	3.10	Langage principal	Développement
NLTK	–	Lemmatisation	NLP

Architecture d'Implémentation et Orchestration

7.1 Vue d'ensemble

Ce chapitre décrit le passage du projet d'une phase exploratoire (notebooks Jupyter) à un système distribué, reproductible et fluxcompatible. L'architecture combine des traitements batch (préparation, entraînement, évaluation) et un traitement streaming (inférence en temps réel), le tout orchestré au sein d'un écosystème conteneurisé. Cinq composants forment le pipeline : les jobs Spark de prétraitement, d'entraînement, d'évaluation et de prédiction streaming, ainsi qu'un producteur Kafka simulant l'arrivée continue d'avis.

7.2 Stratégie de Soumission au Cluster Spark

Les scripts Spark sont embarqués dans des conteneurs légers qui font office de *driver* et soumettent le travail au cluster externe via `spark-submit`. Cette séparation décharge toute la charge de calcul sur les workers, permet de paramétrer finement les ressources (curs, mémoire, partitions de shuffle) et garantit que les exécuteurs héritent de l'environnement Python adéquat (NLTK, etc.). Chaque job crée puis arrête proprement sa `SparkSession`, maîtrisant ainsi le cycle de vie des ressources.

7.3 Philosophie d'Implémentation et Traçabilité

Le pipeline NLP applique une stricte séparation état / nonétat. En batch, les modèles stateful (`HashingTFModel`, `IDFModel`) sont ajustés et sauvegardés ; en streaming, les étapes sans état (tokenisation, lemmatisation, bigrammes) sont reconstruites à l'identique tandis que les modèles stateful sont rechargés, assurant la cohérence exacte de l'espace de features.

Chaque job batch ne produit pas seulement des modèles ou des partitions vectori-

sées, mais aussi des métadonnées JSON (`training_meta.json`, `model_insights.json`). Cette traçabilité (horodatage, hyperparamètres, métriques) permet d’auditer le cycle de vie du modèle et de faciliter les retours arrière en production.

7.4 Le Producteur Kafka : Simulation du Flux Temps Réel

Le script `test_data_kafka_producer.py` lit le jeu de test et envoie les avis un par un dans le topic `reviews.raw`, avec un délai configurable (par défaut 0,8 seconde). Cette simulation réaliste valide la capacité du streaming à consommer des micro-batches, à persister progressivement dans MongoDB et à gérer correctement le topic Kafka.

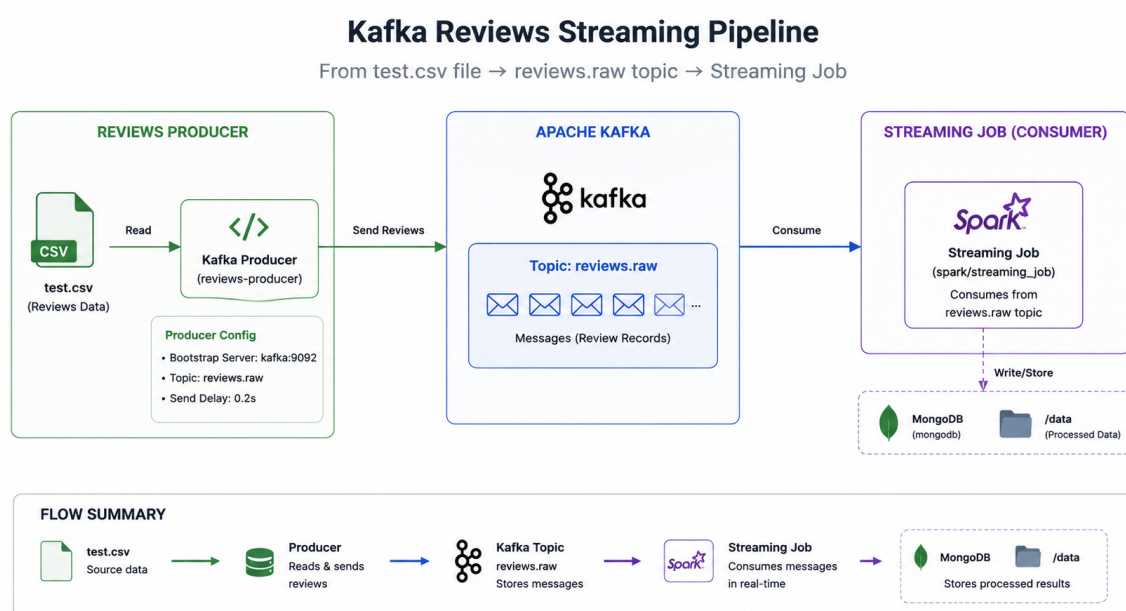


FIGURE 7.1. Logs du producteur Kafka injectant les avis de test.

7.5 Jobs de Préparation et d’Apprentissage (Batch)

7.5.1 Job de Prétraitement

Le *Preprocessing Job* transforme les données brutes en features vectorielles. Après ingestion et labellisation (avec poids de classe amplifiés), les données sont divisées de manière stratifiée (80/10/10). Le pipeline NLP complet est ajusté sur l’entraînement (tokenisation, lemmatisation, bigrammes, TFIDF sur 100 000 dimensions) et produit les partitions Parquet ainsi que les modèles HashingTF et IDF sauvés.

7.5.2 Job d'Entraînement

Training Job lit directement les features Parquet, évitant tout recalcul NLP. Il instancie une Régression Logistique avec les hyperparamètres optimaux identifiés précédemment (ElasticNet, `regParam = 0,001`, etc.) et lance un `fit` distribué. Le modèle est sauvegardé, accompagné de `training_meta.json` qui fige l'horodatage et les paramètres.

7.5.3 Job d'Évaluation

L'*Evaluation Job* charge les features de test, le modèle entraîné et les métadonnées pour assurer la continuité de la lignée. Il applique `model.transform()` puis distribue le calcul des métriques globales et par classe via le `MulticlassClassificationEvaluator` de Spark. Les résultats et la matrice de confusion sont consignés dans `model_insights.json`, certificat de performance avant mise en production.

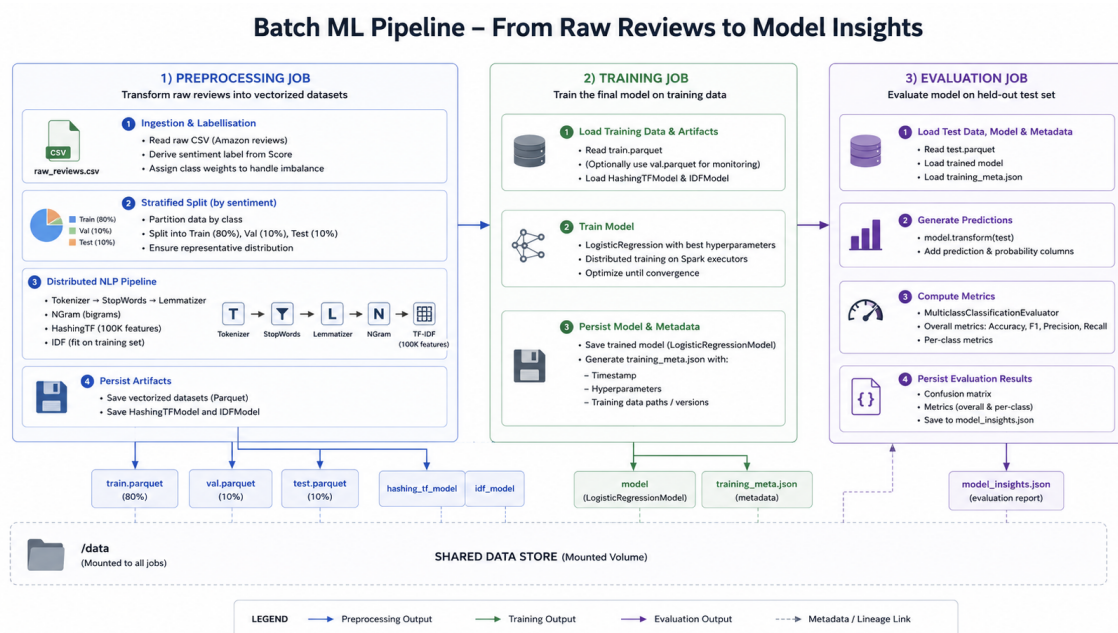


FIGURE 7.2. Vue d'ensemble du pipeline d'entraînement et d'évaluation.

7.6 Job d'Inférence en Flux Continu (Streaming)

Le *Streaming Predict Job* tourne en continu, abonné au topic `reviews.raw` via Spark Structured Streaming. Les messages JSON sont désérialisés selon un schéma explicite, puis le pipeline NLP est reconstruit : les étapes sans état (tokenisation, négations préservées, lemmatisation, bigrammes) sont recréées à l'identique, tandis que l'IDF et le modèle logistique sont chargés depuis le stockage. Les prédictions sont ensuite corrigées par les seuils calibrés avant écriture dans MongoDB. La tolérance aux pannes

repose sur les checkpoints Kafka.

7.7 Orchestration des Pipelines avec Apache Airflow

Apache Airflow orchestre l'ensemble du cycle de vie en modélisant les enchaînements sous forme de DAGs.

7.7.1 Pipeline d'Entraînement End-to-End

Le DAG `train_logistic_regression_model` (déclenchement manuel ou événementiel, `schedule=None`) enchaîne les trois jobs batch via `SparkSubmitOperator`. La séquence `run_preprocessing` » `run_training` » `run_evaluation` garantit la reproductibilité complète de la phase d'apprentissage.

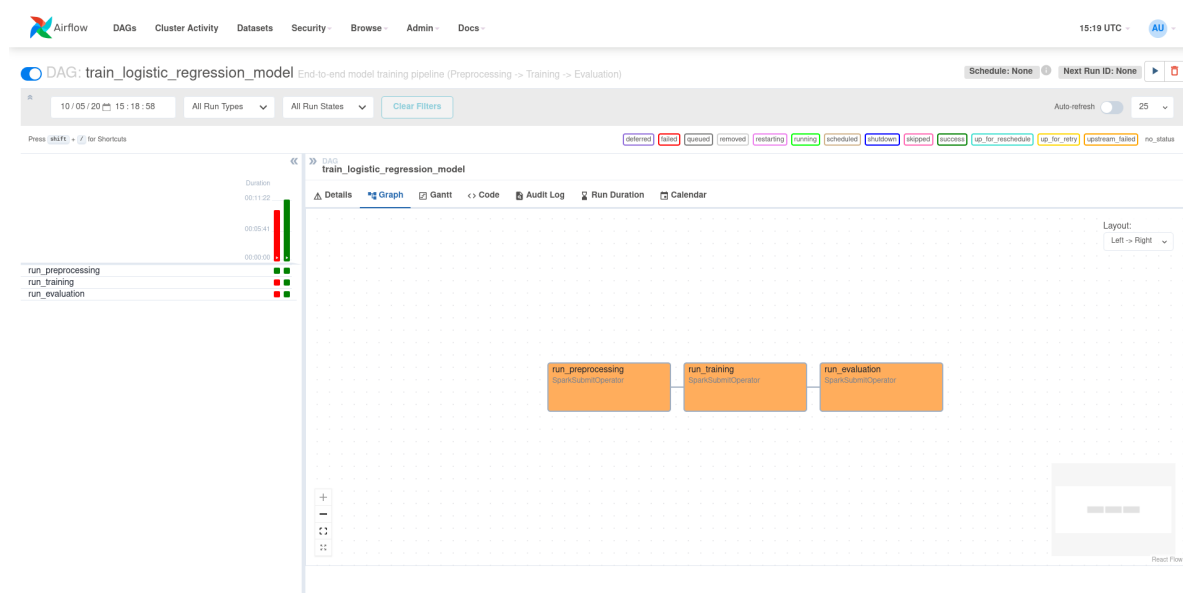


FIGURE 7.3. DAG d'entraînement complet (prétraitement entraînement évaluation).

7.7.2 Agrégation Nocturne pour le Tableau de Bord

Le DAG `dashboard_aggregation` (planifié chaque nuit à 2h) précalcule les agrégats afin que l'API FastAPI ne sollicite pas MongoDB à chaque requête. Deux tâches indépendantes remplacent les collections cibles : une agrégation mensuelle (sentiment par mois) et un scoring par produit (distribution des sentiments par ProductId).

7.7.3 Surveillance du Modèle et Détection de Dérive

Un DAG hebdomadaire (`model_evaluation`, chaque lundi à 8h) surveille la santé du modèle. Une première tâche analyse la distribution actuelle des prédictions dans MongoDB et lève une alerte si le ratio de prédictions *neutre* dépasse 30% ou celui des

negatives 40%. En parallèle, le job Spark d'évaluation est relancé pour comparer les métriques théoriques. Les statuts sont enregistrés dans la collection `model_drift_status`, offrant une vue complète pour décider d'un éventuel réentraînement.

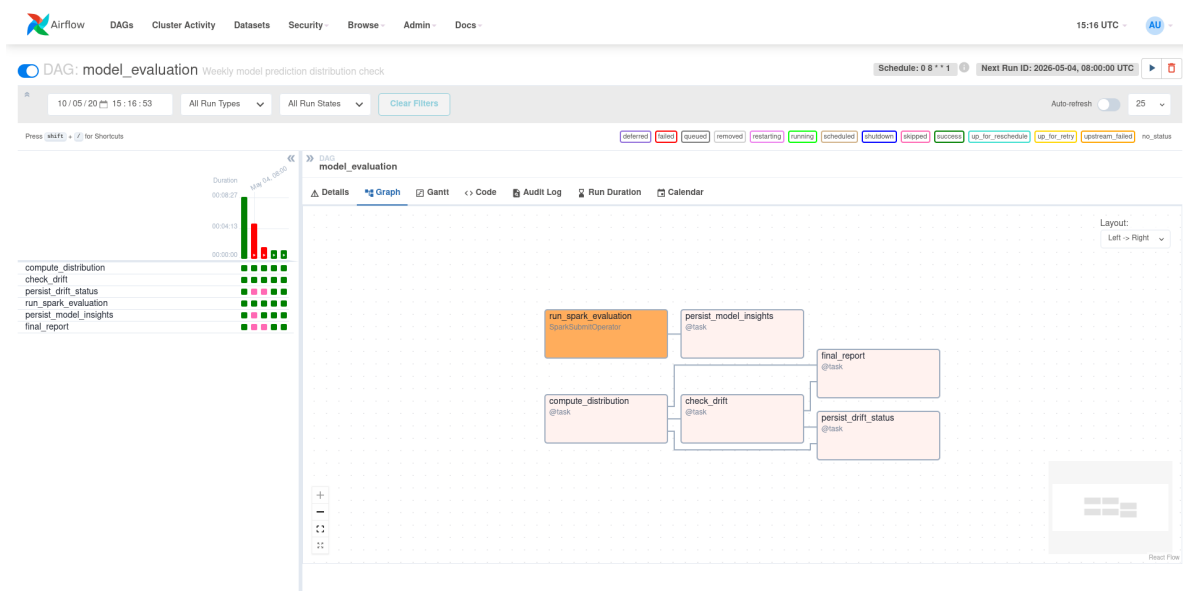


FIGURE 7.4. DAG de surveillance hebdomadaire du modèle.

Intégration Système et Déploiement Local

8.1 Vue d'ensemble

L'ensemble de l'écosystème (Kafka, Spark, MongoDB, Airflow, API FastAPI) est conteneurisé via Docker Compose, garantissant reproductibilité et isolation. Les conteneurs communiquent sur un réseau virtuel dédié, avec volumes partagés pour les données, les modèles et les checkpoints.

8.2 Architecture Conteneurisée et Réseau

Le cluster Spark (master + workers) exécute les traitements batch et streaming. Kafka et Zookeeper assurent le tampon de messages entre producteur et consommateur. MongoDB stocke prédictions, agrégats et métadonnées. Airflow soumet les jobs Spark en accédant aux scripts et données montés en volume. Les principaux volumes incluent :

- /opt/spark/work-dir/data : CSV, Parquet, modèles, JSON de métriques.
- Scripts Spark, montés pour spark-submit.
- Volume dédié aux checkpoints de streaming.

8.3 Couche d'Exposition des Données (API)

Une API REST légère construite avec FastAPI interroge directement les collections MongoDB préagrégées par le DAG dashboard_aggregation (stratégie *Computeonce, Readmany*). Les endpoints exposent :

- les tendances mensuelles (agg_monthly_sentiments),
- le scoring par produit (agg_product_scoring),
- l'état de dérive du modèle (model_drift_status).

Aucune agrégation lourde n'est effectuée à la volée, ce qui garantit des réponses rapides.

8.4 Validation Fonctionnelle

Un test de bout-en-bout valide l'ensemble du pipeline :

1. Démarrage des services avec `docker-compose up`.
2. Déclenchement manuel du DAG `train_logistic_regression_model` qui exécute séquentiellement les jobs Spark jusqu'à générer `model_insights.json`.
3. Lancement du job Spark Streaming à l'écoute du topic `reviews.raw`.
4. Injection d'un lot d'avis par le DAG horaire `amazon_reviews_producer`; les prédictions sont calculées en continu et écrites dans MongoDB.
5. Une requête à l'API FastAPI confirme la présence des nouvelles prédictions, attestant la fluidité du pipeline de la source à l'exposition.

Résultats et Validation du Système

Ce chapitre valide le fonctionnement de bout en bout de l'architecture au travers de captures d'écran des différents services : infrastructure conteneurisée, orchestration Airflow, exécutions Spark, persistance MongoDB et tableau de bord.

9.1 Infrastructure Conteneurisée

La commande `docker ps` confirme que l'ensemble des conteneurs (Spark Master/Workers, Kafka, Zookeeper, MongoDB, Airflow) sont actifs et accessibles sur leurs ports respectifs (figure 9.1).

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
d5ac69b637ed	analyse-avis-clients-temps-produits-amazon-airflow	airflow	"/usr/bin/dumb-init ..."	21 hours ago	Up 2 hours	0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
db8f79280346	provectuslabs/kafka-ui:latest	kafka-ui	"/bin/sh -c 'java --...'"	21 hours ago	Up 17 minutes	0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp
da7afac19ef2	confluentinc/cp-kafka:7.6.0	kafka	"/etc/confluent/dock..."	21 hours ago	Up 4 minutes	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp, 0.0.0.0:9094->9094/tcp, [::]:9094->9094/tcp
f3f22409a262	analyse-avis-clients-temps-produits-amazon-spark-worker-1	spark-worker-1	"/opt/entrypoint.sh ..."	21 hours ago	Up 17 minutes	7077/tcp, 8080/tcp, 0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
b0b9b5efcd84	analyse-avis-clients-temps-produits-amazon-spark-worker-2	spark-worker-2	"/opt/entrypoint.sh ..."	21 hours ago	Up 17 minutes	7077/tcp, 8080/tcp, 0.0.0.0:8082->8081/tcp, [::]:8082->8081/tcp
980ce6dcee50	mongo-express:latest	mongo-express	"/sbin/tini -- /dock..."	21 hours ago	Up 17 minutes	0.0.0.0:9090->8081/tcp, [::]:9090->8081/tcp
04c497b376ed	analyse-avis-clients-temps-produits-amazon-spark-master	spark-master	"/opt/entrypoint.sh ..."	21 hours ago	Up 17 minutes	0.0.0.0:4040->4040/tcp, [::]:4040->4040/tcp, 0.0.0.0:7077->7077/tcp, [::]:7077->7077/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 8081/tcp
8d66cd9b3496	confluentinc/cp-zookeeper:7.6.0	zookeeper	"/etc/confluent/dock..."	21 hours ago	Up 17 minutes	2888/tcp, 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp, 3888/tcp
2e2861204dc8	bitnami/mongodb:latest	mongodb	"/opt/bitnami/script..."	21 hours ago	Up 17 minutes	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp

FIGURE 9.1. Conteneurs opérationnels listés par Docker.

9.2 Orchestration Airflow

L'interface Airflow affiche les quatre DAGs du projet : entraînement manuel, injection horaire, agrégation nocturne et surveillance hebdomadaire (figure 9.2). Le graphe d'exécution du DAG `model_evaluation` illustre l'enchaînement réussi des tâches de détection de dérive et d'évaluation Spark (figure 9.3). Le DAG d'entraînement enchaîne les jobs de prétraitement, entraînement et évaluation (figure 9.4).

9.3 Suivi Spark

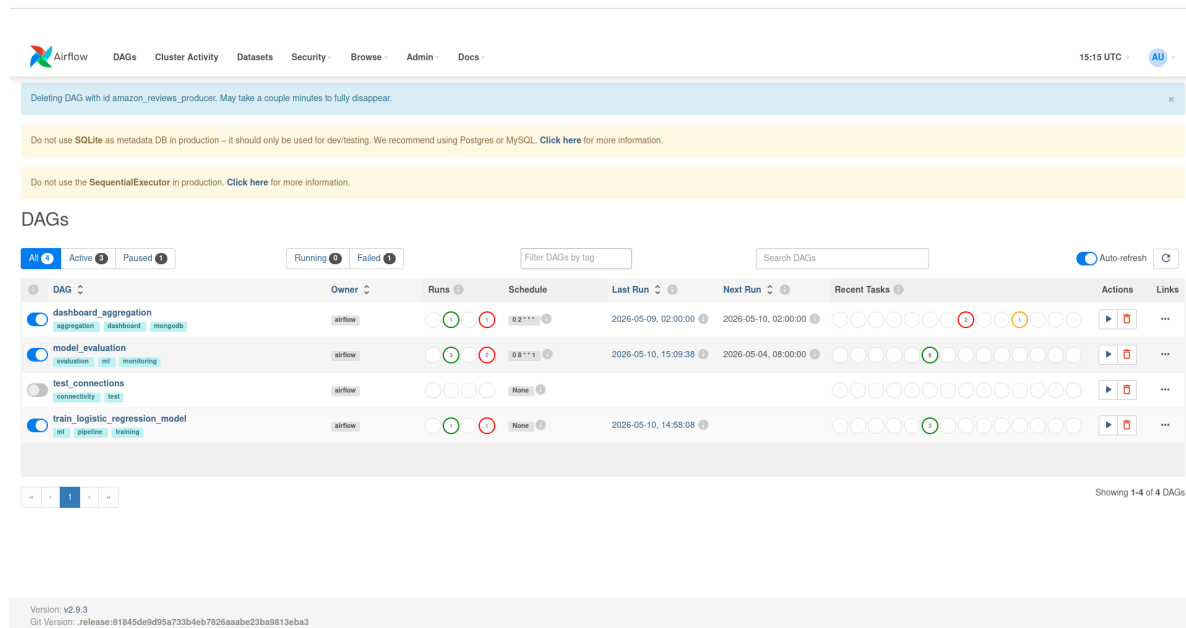


FIGURE 9.2. Liste des DAGs dans Airflow.

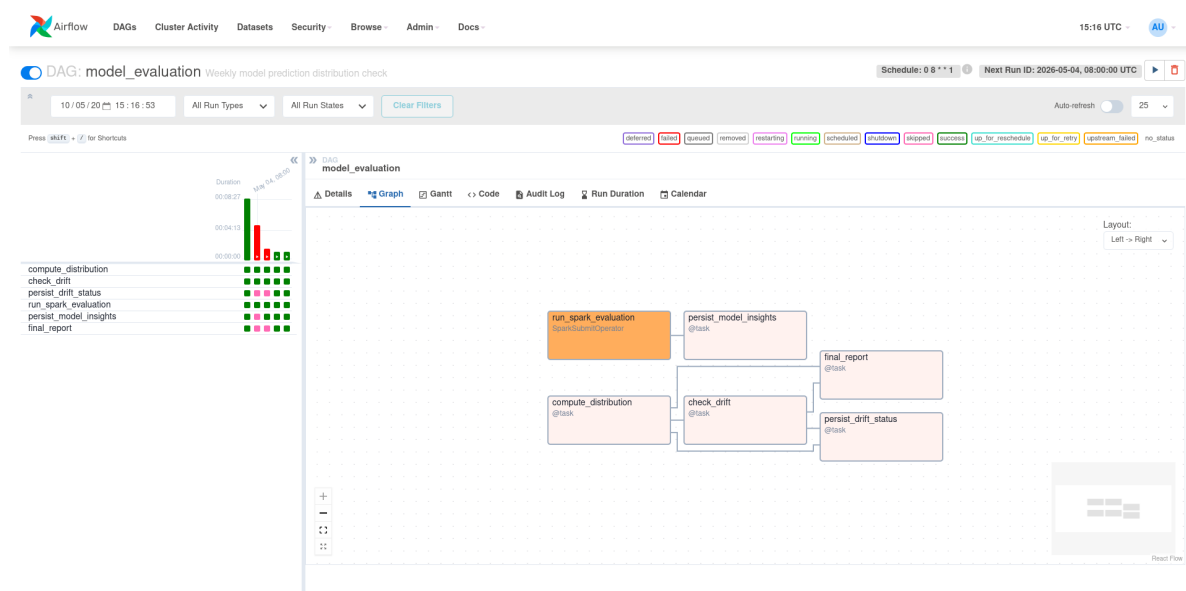


FIGURE 9.3. Graphe du DAG de surveillance hebdomadaire.

L'interface Spark Master montre les applications terminées (entraînement, streaming) et les workers actifs (figure 9.5). Les logs du driver Spark Streaming confirment le traitement batch par batch : ingestion Kafka, transformation NLP, inférence et écriture MongoDB (figure 9.6).

9.4 Persistance MongoDB

La base amazon_reviews contient les collections predictions, agg_monthly_sentiments, agg_product_scoring et model_drift_status (figure 9.7). Un document de la collec-

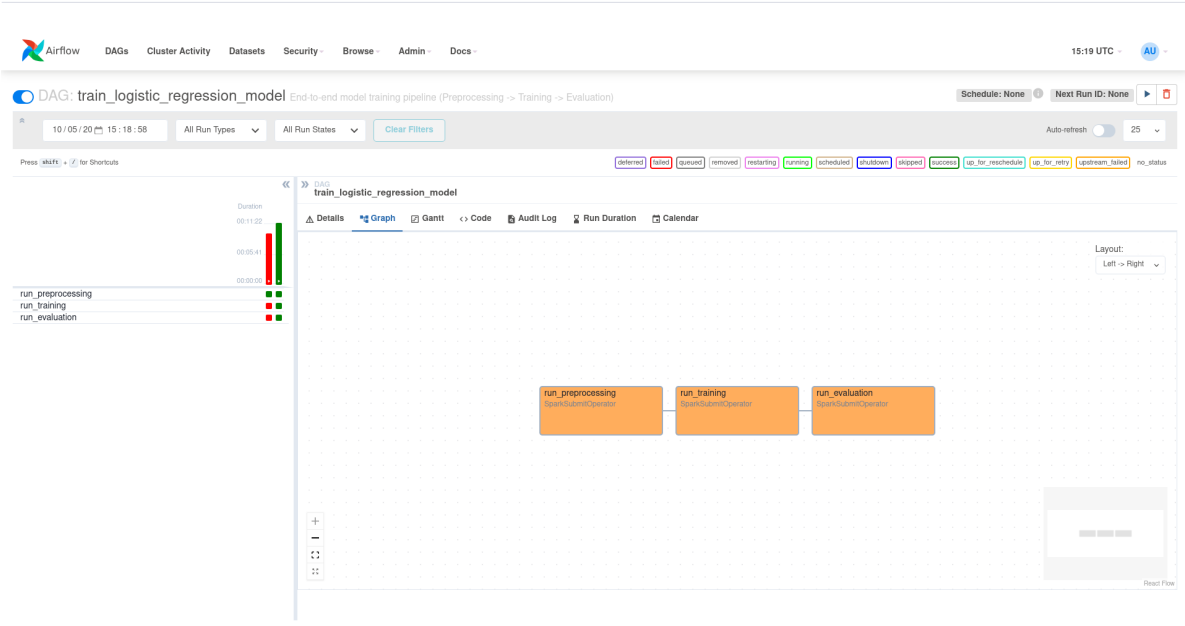


FIGURE 9.4. Graphe du DAG d'entraînement du modèle.

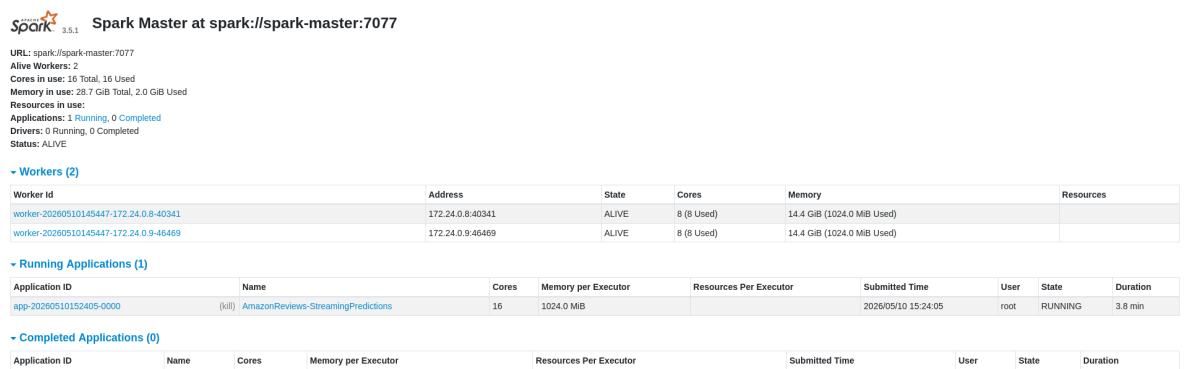


FIGURE 9.5. Tableau de bord Spark Master.

tion predictions montre les champs complets (texte, prédiction, label calibré) (figure 9.8). La collection model_drift_status stocke les ratios de prédictions et le booléen drift_detected, attestant la surveillance automatisée (figure 9.9).

9.5 Tableau de Bord

L'API FastAPI alimente le dashboard qui affiche la répartition des sentiments en temps réel, les agrégations mensuelles, les métriques du modèle et les prédictions récentes (figures 9.10 à 9.14).

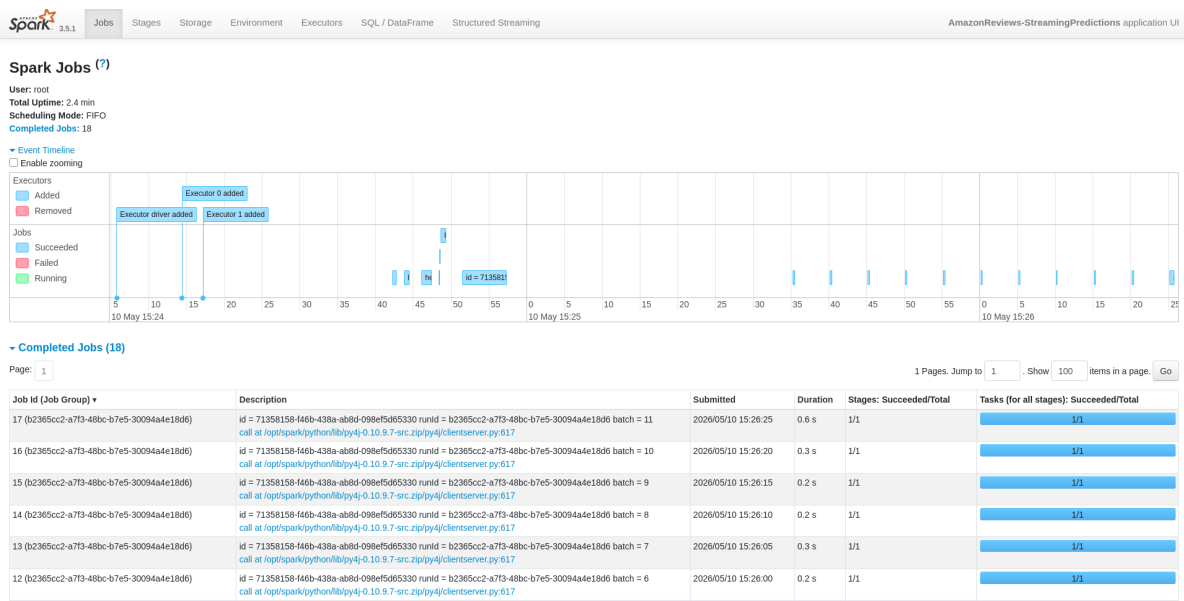


FIGURE 9.6. Logs du job de streaming en direct.

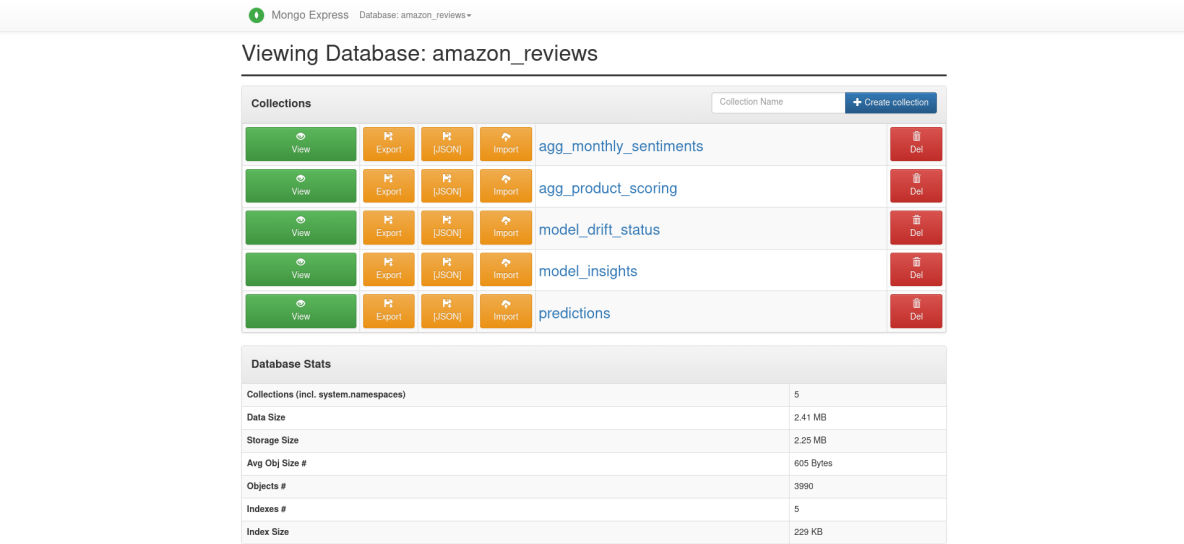


FIGURE 9.7. Collections de la base MongoDB.



FIGURE 9.8. Document de prédiction stocké.

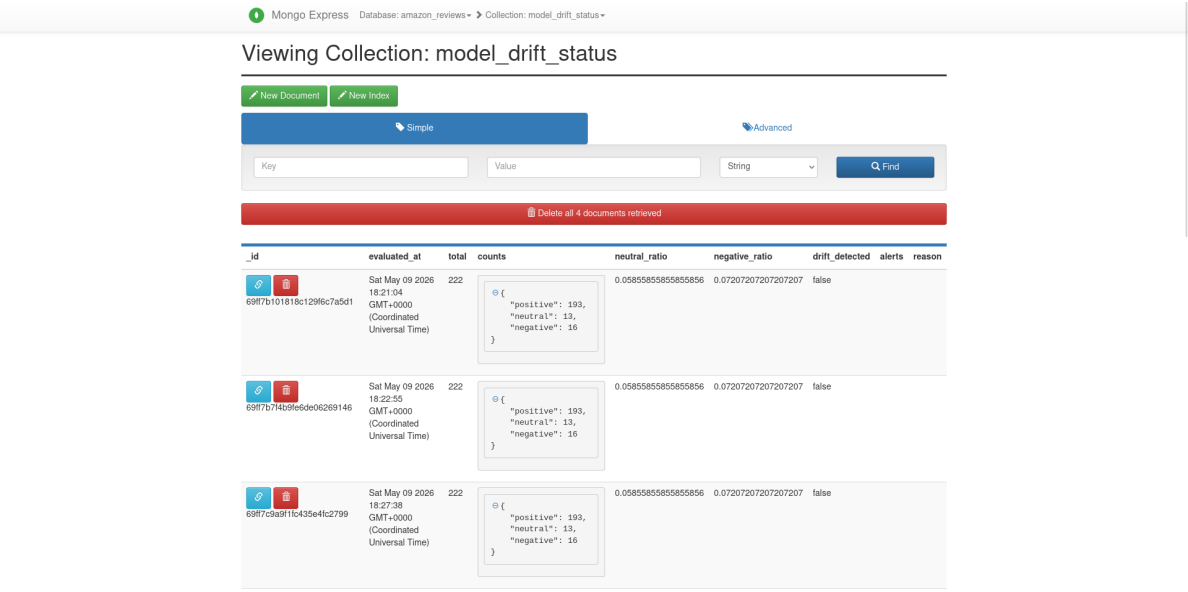


FIGURE 9.9. Suivi de la dérive du modèle.

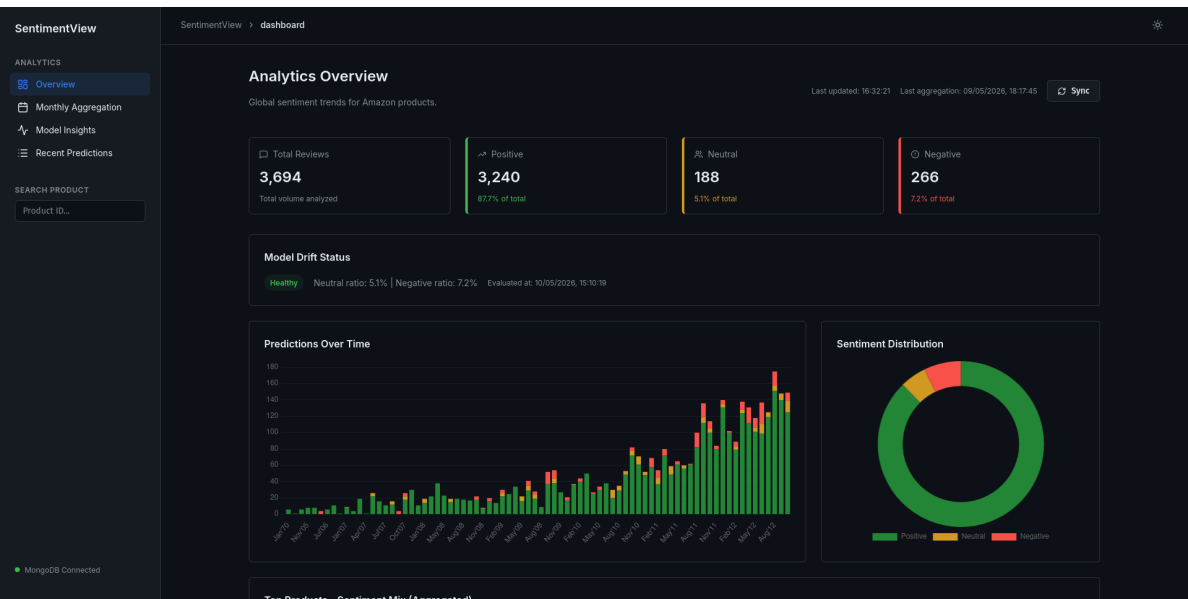


FIGURE 9.10. Vue générale du tableau de bord.

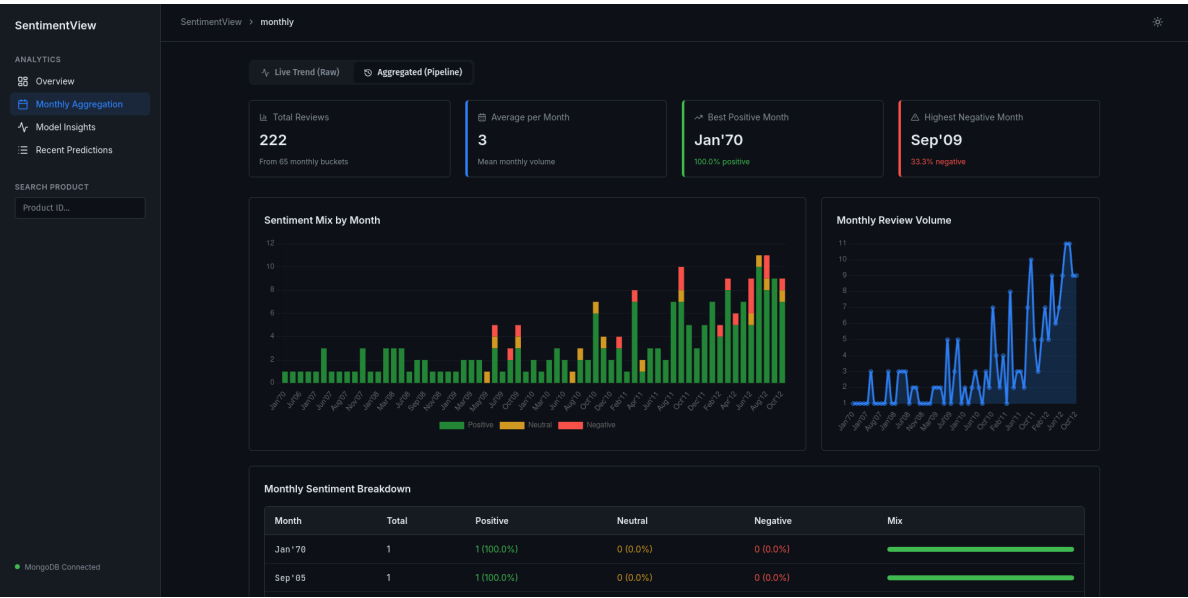


FIGURE 9.11. Agrégation des sentiments par mois.

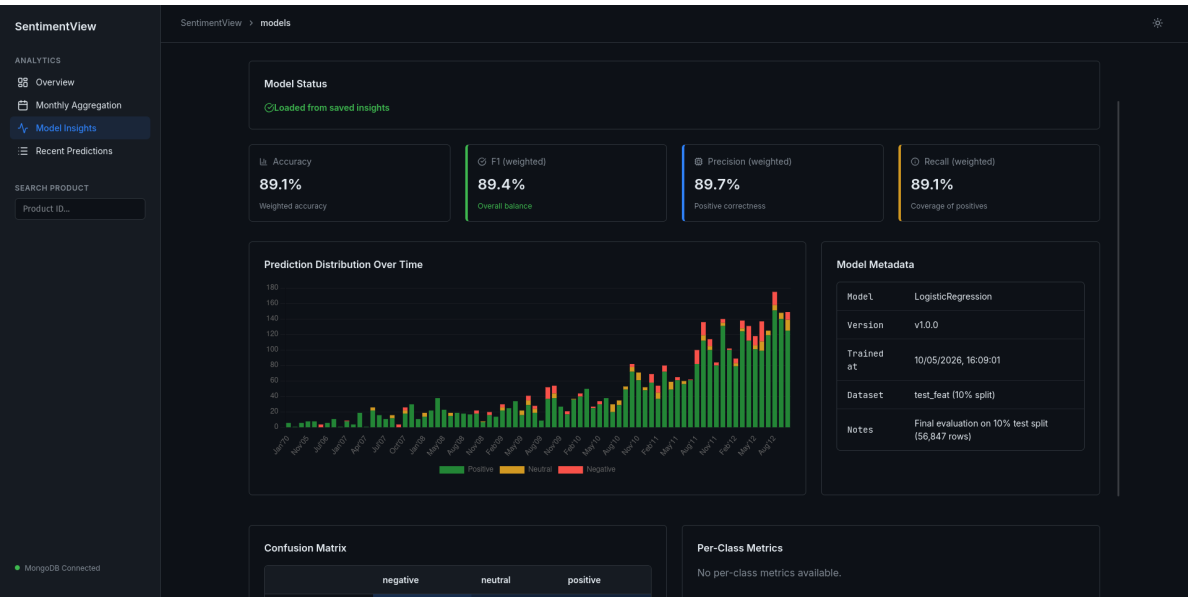


FIGURE 9.12. Informations sur le modèle.

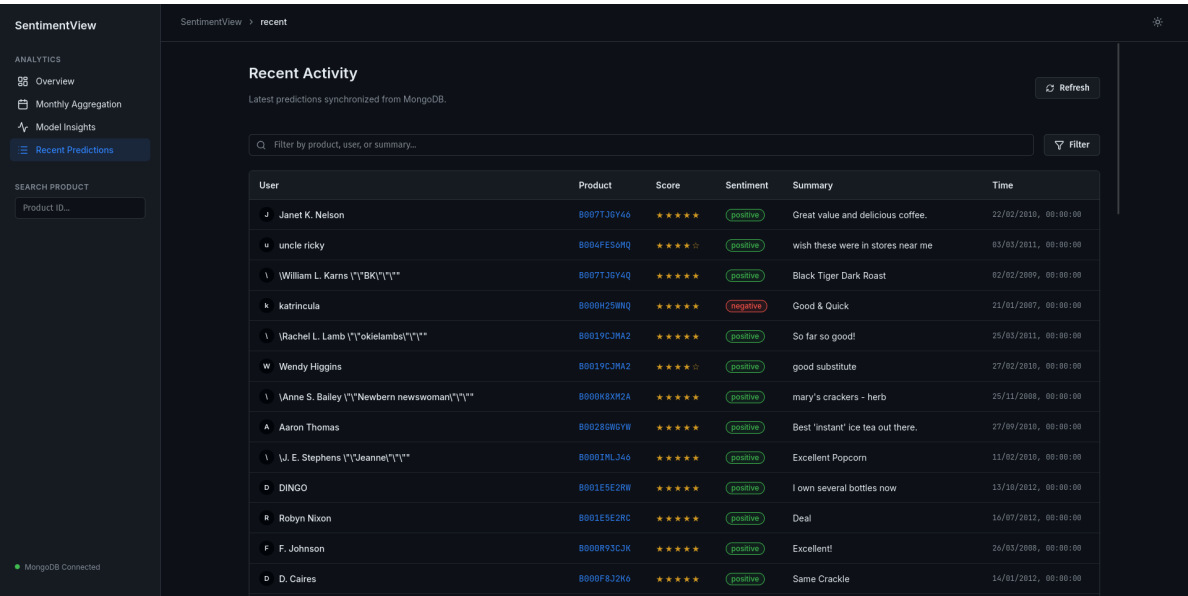


FIGURE 9.13. Prédictions récentes.

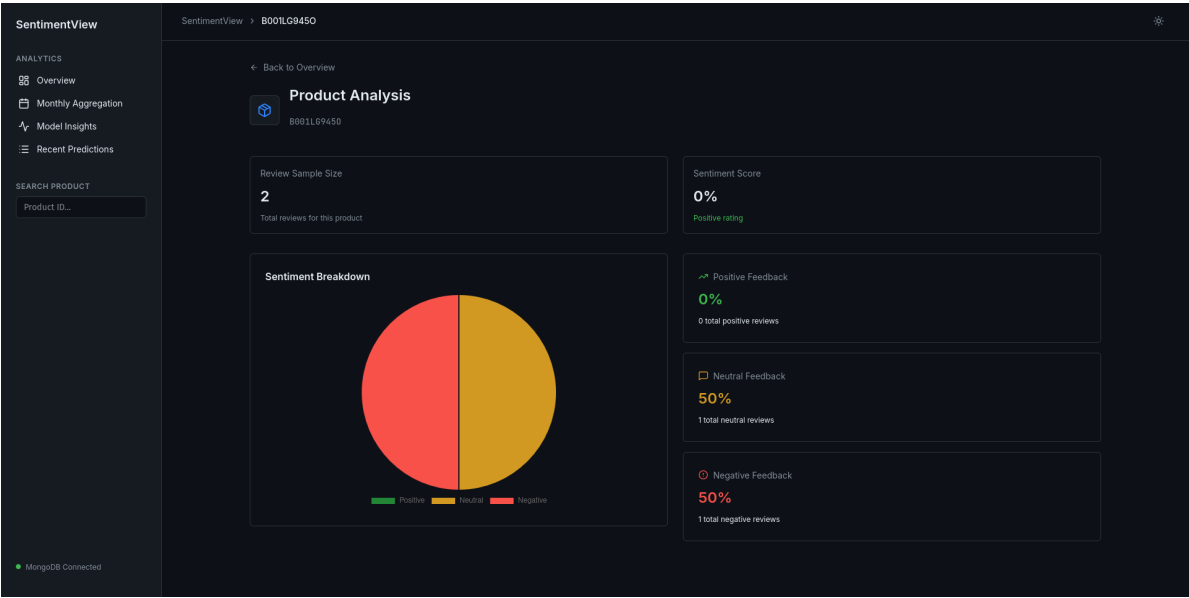


FIGURE 9.14. Performance d'un produit spécifique.

Conclusion

10.1 Bilan du Projet

Ce projet a démontré la faisabilité et la pertinence d'une architecture de Data Engineering complète pour la classification de sentiments sur un jeu de données massif d'avis clients. En partant d'une expérimentation locale menée via des notebooks Jupyter, nous avons réussi à industrialiser un pipeline de Machine Learning bout-en-bout, capable de passer de l'entraînement historique à l'inférence en temps réel.

Le choix d'une architecture hybride, combinant le traitement par lots (Batch) pour la préparation des features et l'entraînement, et le traitement en flux (Streaming) pour la prédiction, s'est avéré être la solution la plus robuste. La phase de batch garantit la reproductibilité et la stabilité de l'espace vectoriel TF-IDF, tandis que la phase de streaming, orchestrée par Spark Structured Streaming et Apache Kafka, assure une réactivité et une scalabilité indispensables pour les cas d'usage industriels.

Du point de vue de la modélisation, l'approche pragmatique adoptée reposant sur la fusion de caractéristiques textuelles (*Summary* et *Text*), l'amplification des poids de classe pour les minorités, et l'ajustement fin des seuils de décision a permis d'obtenir des performances honorables avec un modèle de Régression Logistique, tout en maintenant une interprétabilité et un coût de calcul maîtrisés par rapport à des modèles profonds.

Enfin, l'intégration de l'écosystème au sein de conteneurs Docker, orchestrés par Apache Airflow et communiquant via des réseaux et volumes partagés, a transformé une preuve de concept en un système reproductible, automatisé et prêt à être déployé. La mise en place d'un suivi de dérive en production (*data drift*) et d'agrégations nocturnes pré-calculées vient parachever cette architecture en lui conférant les caractéristiques d'un système MLOps mature.

10.2 Limites Identifiées

Malgré les fonctionnalités opérationnelles du système, plusieurs limites ont été identifiées au cours de ce projet :

- **Performances du modèle sur la classe neutre** : Bien que les poids de classe amplifiés et la calibration des seuils aient amélioré le rappel de la classe neutre, sa précision et son F1-score restent faibles ($F1 \approx 0,23$ sur le jeu de test). La frontière sémantique entre un avis neutre et un avis positif/négatif reste poreuse pour un modèle linéaire.
- **Complexité du pipeline NLP en streaming** : La recreation "à la volée" des étapes de tokenisation, lemmatisation et bigrammes dans le job de streaming, bien que fonctionnelle, induit une latence et une consommation de calcul importantes. Le recours à des UDF Python pour la lemmatisation brise en partie l'optimisation native de la JVM de Spark.
- **Contraintes de la persistance MongoDB** : L'absence de connecteur natif robuste pour Spark Structured Streaming offrant des garanties *exactly-once* a nécessité le recours au patron `foreachBatch`. Bien que fonctionnel, ce mécanisme délègue la gestion transactionnelle à la logique applicative, ce qui peut être risqué en cas de redémarrage intempestif du driver.

10.3 Perspectives d'Amélioration

Pour adresser ces limites et faire évoluer le système vers une plateforme de production à grande échelle, plusieurs axes d'amélioration peuvent être envisagés :

1. **Évolution vers des modèles séquentiels profonds** : Pour mieux capturer les nuances sémantiques et la structure séquentielle du langage, l'intégration de modèles basés sur l'architecture Transformer (tels que DistilBERT ou RoBERTa) via la librairie Spark NLP permettrait d'améliorer significativement la détection de la classe neutre, au prix d'un coût de calcul plus élevé.
2. **Optimization du prétraitement NLP** : La migration de la lemmatisation et de la gestion des bigrammes vers les transformateurs natifs de Spark NLP (qui s'exécutent directement sur la JVM) permettrait d'éliminer le recours aux UDF Python, réduisant drastiquement la latence du pipeline de streaming.
3. **Déploiement sur le Cloud et Scalabilité** : Le passage d'une architecture Docker locale vers un environnement Cloud (AWS, GCP ou Azure) constitue l'étape logique suivante. L'utilisation de services managés (Amazon MSK pour Kafka, EMR pour Spark, Atlas pour MongoDB) permettrait une mise à l'échelle automatique (*auto-scaling*) et une haute disponibilité.

4. **Enrichissement de la couche MLOps** : L'intégration d'un outil dédié au suivi des expériences et des modèles, tel que MLflow, permettrait de versionner les modèles de manière plus granulaire, de comparer les courbes d'apprentissage et d'automatiser le déploiement du modèle en production (*CI/CD pour le Machine Learning*).

En définitive, ce projet constitue une fondation solide, démontrant que l'ingénierie des données et l'automatisation sont tout aussi critiques que l'algorithme de prédiction lui-même pour concevoir des systèmes intelligents fiables et pérennes.